

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ Án Đa Ngành - Hướng Công Nghệ Phần Mềm

BÁO CÁO TỔNG HỢP

Hệ thống IOT Smart Home

GVHD: ThS. Nguyễn Hữu Hiếu

Lớp: L03 - Tên nhóm: 51

Khoa: Khoa học và Kỹ thuật Máy tính

Thành phố Hồ Chí Minh, tháng 3 năm 2026



Danh sách nhóm

STT	MSSV	HỌ	TÊN	NHIỆM VỤ
1	2310856	Trần Anh	Hào	100%
2	2311982	Lê Hoàng	Luân	100%
3	2313132	Nguyễn Tuấn	Thành	100%
4	2313493	Nguyễn Thiện	Toàn	50%



Mục lục

1 Đặc tả hệ thống

1.1 Bối cảnh

Trong bối cảnh Internet of Things (IoT) được ứng dụng ngày càng nhiều trong quản lý nhà ở, hệ thống nhà thông minh cần đáp ứng đồng thời ba yêu cầu: giám sát môi trường, điều khiển thiết bị và truy vết quá trình vận hành. Một hệ thống chỉ bật/tắt thiết bị đơn lẻ chưa đủ để phản ánh bài toán thực tế, vì người dùng còn cần biết dữ liệu cảm biến đến từ đâu, hệ thống tự động ra quyết định như thế nào và các thay đổi đã được ghi nhận ra sao.

Dự án Smart House được xây dựng nhằm quản lý một mô hình nhà thông minh mẫu. Hệ thống tập trung vào dashboard web, backend API, cơ sở dữ liệu PostgreSQL, gateway thiết bị, mô phỏng thiết bị và dịch vụ camera nhận diện người. Các luồng chính gồm đăng nhập, phân quyền, quản lý nhà và thành viên, giám sát cảm biến, điều khiển quạt/đèn/chế độ, cấu hình ngưỡng tự động, lập lịch, cảnh báo, lịch sử dữ liệu và nhật ký audit.

1.2 Các bên liên quan (stakeholders)

Hệ thống Smart House bao gồm các bên liên quan chính sau:

- **Chủ nhà / thành viên trong nhà**
 - Theo dõi dữ liệu môi trường như nhiệt độ, độ ẩm, ánh sáng và chuyển động.
 - Điều khiển thiết bị thuộc nhà được cấp quyền, gồm quạt, đèn và chế độ vận hành.
 - Xem lịch sử dữ liệu, trạng thái thiết bị và cảnh báo phát sinh.
- **Quản trị viên**
 - Quản lý tài khoản, quyền truy cập và nhật ký hệ thống.
 - Cấu hình ngưỡng tự động, lịch chế độ và hồ sơ vận hành của nhà.
 - Theo dõi hoạt động audit để kiểm tra thay đổi cấu hình, cảnh báo và thao tác điều khiển.
- **Kỹ thuật viên / bên lắp đặt**
 - Theo dõi trạng thái thiết bị và các cảnh báo liên quan đến kết nối hoặc cảm biến.
 - Hỗ trợ xử lý cảnh báo và bảo trì thiết bị khi có sự cố.
- **Thiết bị IoT và gateway**
 - Gửi telemetry từ các cảm biến lên backend.
 - Nhận lệnh điều khiển đang chờ và gửi xác nhận sau khi xử lý.
 - Kết nối thêm dịch vụ camera để phục vụ nhận diện người và chuyển động.

1.3 Mục tiêu dự án

- Xây dựng hệ thống Smart House dựa trên mô hình IoT có frontend, backend, database và gateway thiết bị.
- Thu thập dữ liệu từ các cảm biến nhiệt độ, độ ẩm, ánh sáng, chuyển động và các loại cảm biến mở rộng được định nghĩa trong hệ thống.
- Quản lý thiết bị theo mô hình capability, trong đó mỗi thiết bị có các năng lực như POWER, SPEED, BRIGHTNESS, MODE, TEMPERATURE, HUMIDITY, MOTION.
- Hỗ trợ điều khiển thủ công, điều khiển tự động theo cấu hình, lịch chạy nền và cơ chế giữ trạng thái thủ công tạm thời.
- Cung cấp dashboard web để hiển thị dữ liệu giám sát, trạng thái thiết bị, cảnh báo và cập nhật realtime bằng Server-Sent Events.
- Lưu trữ dữ liệu cảm biến, trạng thái hiện tại, lịch sử trạng thái, lệnh điều khiển, cảnh báo và nhật ký audit trong PostgreSQL.
- Hỗ trợ xác thực bằng tài khoản cục bộ, Google OAuth, JWT access token và refresh token.
- Tích hợp RabbitMQ cho luồng sự kiện người dùng thông qua bảng outbox, đồng thời hỗ trợ thông báo email/Telegram khi cấu hình được bật.

1.4 Phạm vi dự án

- Frontend web được xây dựng bằng React và Vite, gồm các màn hình đăng nhập, dashboard, lịch sử, cấu hình, cài đặt, audit log và quản lý người dùng.
- Backend được xây dựng bằng Java 17 và Spring Boot, cung cấp RESTful API cho xác thực, thiết bị, telemetry, điều khiển, cấu hình, cảnh báo, lịch, thành viên nhà và audit.
- Cơ sở dữ liệu PostgreSQL được quản lý bằng Flyway migration, sử dụng các bảng thực thể như `users`, `homes`, `devices`, `sensors`, `sensor_data`, `configs`, `control_commands`, `alerts`, `activity_logs`.
- Gateway Python bằng Flask làm lớp trung gian cho thiết bị, hỗ trợ gửi telemetry, lấy lệnh điều khiển, xác nhận lệnh và gọi dịch vụ YOLO.
- Mã thiết bị OhStem/YoloBit và simulator Python phục vụ demo luồng cảm biến, điều khiển, hồng ngoại, cửa, quạt, đèn và fallback cục bộ.
- Dịch vụ camera dùng OpenCV và Ultralytics YOLO để phát hiện người/chuyển động qua endpoint `/check_human`.

2 Các yêu cầu hệ thống

2.1 User Requirements

2.1.1 Module 1 – Nhận và hiển thị dữ liệu từ thiết bị

Mục tiêu: Thu thập dữ liệu cảm biến và hiển thị theo thời gian thực.

- (FR1) Hệ thống phải nhận dữ liệu từ các cảm biến nhiệt độ, độ ẩm, ánh sáng, chuyển động và các loại cảm biến mở rộng được định nghĩa trong backend.
- (FR2) Dashboard Web phải cập nhật dữ liệu mới thông qua API tổng hợp và kênh realtime Server-Sent Events.
- (FR3) Hệ thống phải hiển thị trạng thái hiện tại của các thiết bị theo capability như POWER, SPEED, BRIGHTNESS, MODE.
- (FR4) Người dùng phải xem được biểu đồ lịch sử dữ liệu cảm biến theo khoảng thời gian được hỗ trợ trên giao diện History.
- (FR5) Hệ thống phải lưu trữ dữ liệu cảm biến vào cơ sở dữ liệu để phục vụ việc truy xuất và phân tích sau này.
- (FR6) Nếu thiết bị không gửi dữ liệu trong khoảng thời gian cấu hình, hệ thống phải cập nhật trạng thái offline và tạo cảnh báo phù hợp.

2.1.2 Module 2 – Kiểm tra dữ liệu vượt ngưỡng

Mục tiêu: Phát hiện các điều kiện bất thường và tạo cảnh báo.

- (FR7) Quản trị viên phải có khả năng cấu hình các ngưỡng an toàn như: nhiệt độ cao/thấp, ánh sáng cao/thấp và nhiệt độ nguy hiểm.
- (FR8) Nếu nhiệt độ vượt ngưỡng cao trong khoảng thời gian cấu hình (ví dụ 2 phút), hệ thống phải tạo cảnh báo.
- (FR9) Nếu ánh sáng thấp hơn hoặc cao hơn mức cấu hình, hệ thống phải kích hoạt hành động tự động tương ứng.
- (FR10) Hệ thống phải áp dụng cơ chế hysteresis để tránh việc thiết bị bật/tắt liên tục khi giá trị cảm biến dao động quanh ngưỡng.
- (FR11) Khi giá trị vượt ngưỡng nguy hiểm (ví dụ $T_{critical}$), hệ thống phải hiển thị cảnh báo trên dashboard, hỗ trợ thông báo qua kênh được cấu hình và ghi log sự kiện.
- (FR12) Nếu phát hiện dữ liệu cảm biến không hợp lệ hoặc vượt ngoài miền cho phép, hệ thống phải chuyển thiết bị về trạng thái an toàn và ghi nhận lỗi.
- (FR13) Mỗi cảnh báo phải có ba trạng thái: *Active*, *Acknowledged* và *Resolved*.

2.1.3 Module 3 – Điều khiển thiết bị

Mục tiêu: Cho phép người dùng điều khiển thiết bị theo cách thủ công hoặc tự động.

- (FR14) Người dùng phải có khả năng điều khiển thiết bị từ dashboard Web và theo dõi trạng thái lệnh.
- (FR15) Thiết bị/simulator hỗ trợ nhập mật khẩu bảo mật qua luồng hồng ngoại; khi nhập sai quá số lần cấu hình, hệ thống tạo cảnh báo và ghi log sự kiện.
- (FR16) Khi người dùng điều khiển thủ công trong chế độ tự động, hệ thống phải chuyển sang chế độ MANUAL, giữ trạng thái thiết bị trong thời gian T_{hold} và sau đó quay lại chế độ trước đó.
- (FR17) Trong chế độ AUTO, hệ thống phải tự động điều khiển quạt theo nhiệt độ và đèn theo mức ánh sáng.
- (FR18) Trong chế độ SLEEP, hệ thống phải ưu tiên tiết kiệm điện và giảm mức hoạt động của thiết bị.
- (FR19) Trong chế độ AWAY, hệ thống phải tắt các thiết bị không cần thiết và chỉ kích hoạt khi vượt ngưỡng an toàn.
- (FR20) Quản trị viên phải có khả năng cấu hình lịch tự động chuyển đổi giữa các chế độ AUTO, SLEEP và AWAY theo khung giờ.

2.1.4 Module 4 – Ghi nhận hoạt động (Logging và Audit)

Mục tiêu: Theo dõi và lưu trữ toàn bộ hoạt động của hệ thống.

- (FR21) Mọi hành động bật/tắt thiết bị phải được ghi log bao gồm: người thực hiện, thời điểm và trạng thái trước/sau của thiết bị.
- (FR22) Mọi cảnh báo phải được lưu lại với các thông tin: thời điểm kích hoạt, thời điểm xác nhận và thời điểm kết thúc.
- (FR23) Mọi thay đổi cấu hình hệ thống như thay đổi ngưỡng, lịch hoạt động hoặc chế độ mặc định phải được ghi log đầy đủ.
- (FR24) Quản trị viên phải có khả năng truy xuất và lọc lịch sử hoạt động theo thời gian, loại sự kiện và người thực hiện.

2.1.5 Module 5 – Ứng dụng Web

Mục tiêu: Cung cấp giao diện sử dụng và phân quyền truy cập.

- (FR25) Hệ thống phải yêu cầu người dùng đăng nhập bằng tài khoản cục bộ hoặc Google OAuth để truy cập các chức năng được bảo vệ.
- (FR26) Hệ thống phải hỗ trợ phân quyền người dùng gồm:
 - ADMIN/SUPER_ADMIN: quản lý người dùng, cấu hình hệ thống và xem audit log.
 - Người dùng trong home: xem dữ liệu, điều khiển thiết bị và thao tác trong phạm vi quyền home.
 - Vai trò home như OWNER, CO_OWNER, RESIDENT, TECHNICIAN, VIEWER quyết định quyền ở cấp nhà.
- (FR27) Dashboard tổng quan phải hiển thị dữ liệu realtime, trạng thái thiết bị và các cảnh báo hiện tại.
- (FR28) Quản trị viên phải có khả năng cấu hình ngưỡng và lịch hoạt động trực tiếp từ giao diện hệ thống.
- (FR29) Người dùng phải xem được cảnh báo mới trên dashboard; các kênh email, Telegram phụ thuộc cấu hình triển khai.

2.2 Yêu cầu chức năng

Danh sách các actor:

- User (Chủ nhà)
- Admin (Quản trị hệ thống)
- Installer (Bên lắp đặt)
- IoT Device (Thiết bị cảm biến & điều khiển)
- System (Hệ thống tự động)

Để hiểu rõ thêm về yêu cầu chức năng của hệ thống nhóm tác giả đã mô hình hóa bằng các sơ đồ use-case sau:

USE CASE 1: ĐĂNG NHẬP HỆ THỐNG

Usecase name	Đăng nhập hệ thống
Description	Người dùng đăng nhập vào hệ thống để truy cập chức năng theo role.
Actors	User / Admin / Installer
Preconditions	Tài khoản hợp lệ tồn tại trong hệ thống.
Trigger	Người dùng nhập username và password.
Post-conditions	Success: Tạo session/token và chuyển đến dashboard. Failure: Không tạo session.
Main flow	1. Nhập thông tin đăng nhập. 2. Hệ thống xác thực. 3. Xác định role. 4. Chuyển hướng dashboard.
Alternative flow	Sai thông tin → Thông báo lỗi.
Exceptions	Lỗi server hoặc database.

USE CASE 2: GỬI DỮ LIỆU CẢM BIẾN

Usecase name	Gửi dữ liệu cảm biến
Description	Thiết bị gửi dữ liệu định kỳ về server.
Actors	IoT Device
Preconditions	Thiết bị đang hoạt động và có kết nối mạng.
Trigger	Đến chu kỳ gửi dữ liệu.
Post-conditions	Dữ liệu được lưu vào database.
Main flow	1. Thu thập dữ liệu. 2. Gửi lên server. 3. Server kiểm tra hợp lệ. 4. Lưu database.
Alternative flow	Dữ liệu không hợp lệ → Bỏ qua và ghi log.
Exceptions	Mất kết nối mạng.

USE CASE 3: XEM DỮ LIỆU REALTIME

Usecase name	Xem dữ liệu realtime
Description	Người dùng xem dữ liệu cập nhật theo thời gian thực.
Actors	User
Preconditions	Đã đăng nhập thành công.
Trigger	Mở dashboard.
Post-conditions	Dữ liệu hiển thị ≤ 2 giây sau khi cập nhật.
Main flow	1. Truy cập dashboard. 2. Hệ thống lấy dữ liệu mới nhất. 3. Hiển thị lên giao diện.
Alternative flow	Không có dữ liệu → Hiển thị trạng thái chờ.
Exceptions	Lỗi truy vấn database.

USE CASE 4: XEM BIỂU ĐỒ LỊCH SỬ

Usecase name	Xem biểu đồ lịch sử
Description	Xem dữ liệu theo 1h, 24h, 7 ngày.
Actors	User
Preconditions	Có dữ liệu lưu trữ.
Trigger	Chọn khoảng thời gian.
Post-conditions	Hiển thị biểu đồ tương ứng.
Main flow	1. Chọn loại dữ liệu. 2. Chọn mốc thời gian. 3. Hệ thống truy vấn. 4. Hiển thị biểu đồ.
Alternative flow	Không có dữ liệu → Thông báo.
Exceptions	Lỗi truy vấn.

USE CASE 5: XEM TRẠNG THÁI THIẾT BỊ

Usecase name	Xem trạng thái thiết bị
Description	Hiển thị ON/OFF hoặc Offline.
Actors	User
Preconditions	Đã đăng nhập.
Trigger	Mở dashboard.
Post-conditions	Trạng thái hiển thị chính xác.
Main flow	1. Hệ thống lấy trạng thái thiết bị. 2. Hiển thị ON/OFF.
Alternative flow	Không phản hồi → Hiển thị Offline.
Exceptions	Mất kết nối thiết bị.

USE CASE 6: CẤU HÌNH NGƯỠNG CẢNH BÁO

Usecase name	Cấu hình ngưỡng cảnh báo
Description	Admin thiết lập giá trị ngưỡng high/low/critical cho cảm biến.
Actors	Admin
Preconditions	Admin đã đăng nhập.
Trigger	Admin truy cập trang cấu hình.
Post-conditions	Ngưỡng mới được lưu vào hệ thống.
Main flow	1. Mở trang cấu hình. 2. Nhập giá trị ngưỡng. 3. Nhấn lưu. 4. Hệ thống kiểm tra hợp lệ. 5. Lưu vào database.
Alternative flow	Giá trị không hợp lệ → Thông báo lỗi.
Exceptions	Lỗi lưu database.

USE CASE 7: PHÁT HIỆN VƯỢT NGƯỠNG

Usecase name	Phát hiện vượt ngưỡng
Description	Hệ thống kiểm tra dữ liệu và phát hiện giá trị bất thường.
Actors	System
Preconditions	Có dữ liệu cảm biến mới.
Trigger	Nhận dữ liệu từ thiết bị.
Post-conditions	Tạo cảnh báo trạng thái Active nếu vượt ngưỡng.
Main flow	1. Nhận dữ liệu mới. 2. So sánh với ngưỡng cấu hình. 3. Kiểm tra thời gian duy trì vượt ngưỡng. 4. Tạo cảnh báo.
Alternative flow	Không vượt ngưỡng → Không tạo cảnh báo.
Exceptions	Lỗi xử lý dữ liệu.

USE CASE 8: GỬI CẢNH BÁO

Usecase name	Gửi cảnh báo
Description	Hệ thống gửi thông báo khi có cảnh báo mới.
Actors	System
Preconditions	Cảnh báo đã được tạo.
Trigger	Trạng thái cảnh báo = Active.
Post-conditions	Người dùng và installer nhận thông báo.
Main flow	1. Hiển thị cảnh báo trên dashboard. 2. Gửi notification/email. 3. Ghi log sự kiện.
Alternative flow	Gửi email thất bại → Ghi log lỗi.
Exceptions	Lỗi dịch vụ gửi email.

USE CASE 9: XÁC NHẬN CẢNH BÁO

Usecase name	Xác nhận cảnh báo
Description	User/Admin xác nhận cảnh báo đang Active và Installer xác nhận Resolved sau khi giải quyết.
Actors	User / Admin / Installer
Preconditions	Có cảnh báo trạng thái Active.
Trigger	Admin hoặc User nhấn nút Acknowledge. Installer nhấn nút Resolved
Post-conditions	Trạng thái chuyển sang Acknowledged khi xác nhận lỗi và chuyển sang Resolved khi đã sửa.
Main flow	1. Chọn cảnh báo. 2. Nhấn Acknowledge/Resolved. 3. Hệ thống cập nhật trạng thái. 4. Ghi log.
Alternative flow	Cảnh báo đã được xử lý trước đó.
Exceptions	Lỗi cập nhật database.

USE CASE 10: ĐIỀU KHIỂN THỦ CÔNG

Usecase name	Điều khiển thiết bị thủ công
Description	User bật/tắt thiết bị từ dashboard.
Actors	User
Preconditions	Thiết bị đang online.
Trigger	Nhấn nút ON/OFF.
Post-conditions	Thiết bị đổi trạng thái thành công.
Main flow	1. User chọn thiết bị. 2. Nhấn ON/OFF. 3. Hệ thống gửi lệnh. 4. Thiết bị phản hồi. 5. Cập nhật giao diện.
Alternative flow	Thiết bị không phản hồi → Thông báo lỗi.
Exceptions	Mất kết nối HTTP/API hoặc thiết bị.

USE CASE 11: XÁC THỰC MẬT KHẨU NHÀ

Usecase name	Xác thực mật khẩu nhà
Description	Kiểm tra quyền truy cập qua mật khẩu tại thiết bị.
Actors	User
Preconditions	Thiết bị hoạt động.
Trigger	Người dùng nhập mật khẩu.
Post-conditions	Đăng nhập thành công.
Main flow	1. Người dùng nhập mật khẩu 2. Hệ thống kiểm tra mật khẩu chính xác. 3. Nếu đúng, cho phép truy cập hệ thống
Alternative flow	Nhập mật khẩu sai → Hệ thống đếm số lần nhập sai → Nếu quá 3 lần sai, hệ thống báo động, gửi cảnh báo cho chủ nhà, ghi log cho sự kiện
Exceptions	Mất kết nối hệ thống hoặc thiết bị.

USE CASE 12: MANUAL OVERRIDE

Usecase name	Manual Override
Description	Chuyển sang chế độ MANUAL khi user điều khiển trong AUTO.
Actors	User
Preconditions	Hệ thống đang ở chế độ AUTO.
Trigger	User thực hiện điều khiển thủ công.
Post-conditions	Sau Thold phút quay lại AUTO.
Main flow	1. Phát hiện lệnh thủ công. 2. Chuyển sang MANUAL. 3. Giữ trạng thái Thold phút. 4. Tự động quay lại AUTO.
Alternative flow	User tiếp tục thao tác → Reset thời gian Thold.
Exceptions	Lỗi timer hệ thống.

USE CASE 13: CHUYỂN CHẾ ĐỘ HOẠT ĐỘNG

Usecase name	Chuyển chế độ hoạt động
Description	Người dùng chuyển đổi giữa AUTO, SLEEP, AWAY.
Actors	User
Preconditions	Đã đăng nhập hệ thống.
Trigger	Chọn chế độ trên dashboard.
Post-conditions	Hệ thống áp dụng rule tương ứng với mode mới.
Main flow	1. Người dùng chọn mode. 2. Hệ thống xác nhận yêu cầu. 3. Cập nhật mode hiện tại. 4. Áp dụng rule điều khiển thiết bị.
Alternative flow	Mode đã được chọn trước đó → Không thay đổi.
Exceptions	Lỗi cập nhật database.

USE CASE 14: LẬP LỊCH CHUYỂN CHẾ ĐỘ

Usecase name	Lập lịch chuyển chế độ
Description	Admin cấu hình lịch tự động chuyển mode theo thời gian.
Actors	Admin
Preconditions	Admin đã đăng nhập.
Trigger	Truy cập trang cấu hình lịch.
Post-conditions	Lịch được lưu và tự động thực thi theo thời gian cấu hình.
Main flow	1. Nhập thời gian bắt đầu/kết thúc. 2. Chọn mode tương ứng. 3. Lưu cấu hình. 4. Hệ thống kích hoạt scheduler.
Alternative flow	Thời gian trùng lặp → Thông báo lỗi.
Exceptions	Lỗi hệ thống scheduler.

USE CASE 15: GHI LOG ĐIỀU KHIỂN

Usecase name	Ghi log điều khiển thiết bị
Description	Hệ thống ghi nhận mọi hành động bật/tắt thiết bị.
Actors	System
Preconditions	Có hành động điều khiển thiết bị.
Trigger	User/Admin thực hiện điều khiển.
Post-conditions	Log được lưu vào database.
Main flow	1. Nhận sự kiện điều khiển. 2. Ghi user, thời gian, trạng thái trước/sau. 3. Lưu database.
Alternative flow	Không có thay đổi trạng thái → Không ghi log.
Exceptions	Lỗi ghi database.

USE CASE 16: GHI LOG CẢNH BÁO

Usecase name	Ghi log cảnh báo
Description	Lưu lại toàn bộ vòng đời của cảnh báo.
Actors	System
Preconditions	Có cảnh báo được tạo.
Trigger	Trạng thái cảnh báo thay đổi.
Post-conditions	Lưu thời điểm Active/Acknowledge/Resolved.
Main flow	- Nhận sự kiện thay đổi trạng thái. - Cập nhật thời điểm tương ứng. - Lưu vào database.
Alternative flow	Không thay đổi trạng thái → Không cập nhật.
Exceptions	Lỗi database.

USE CASE 17: XEM & LỌC LOG

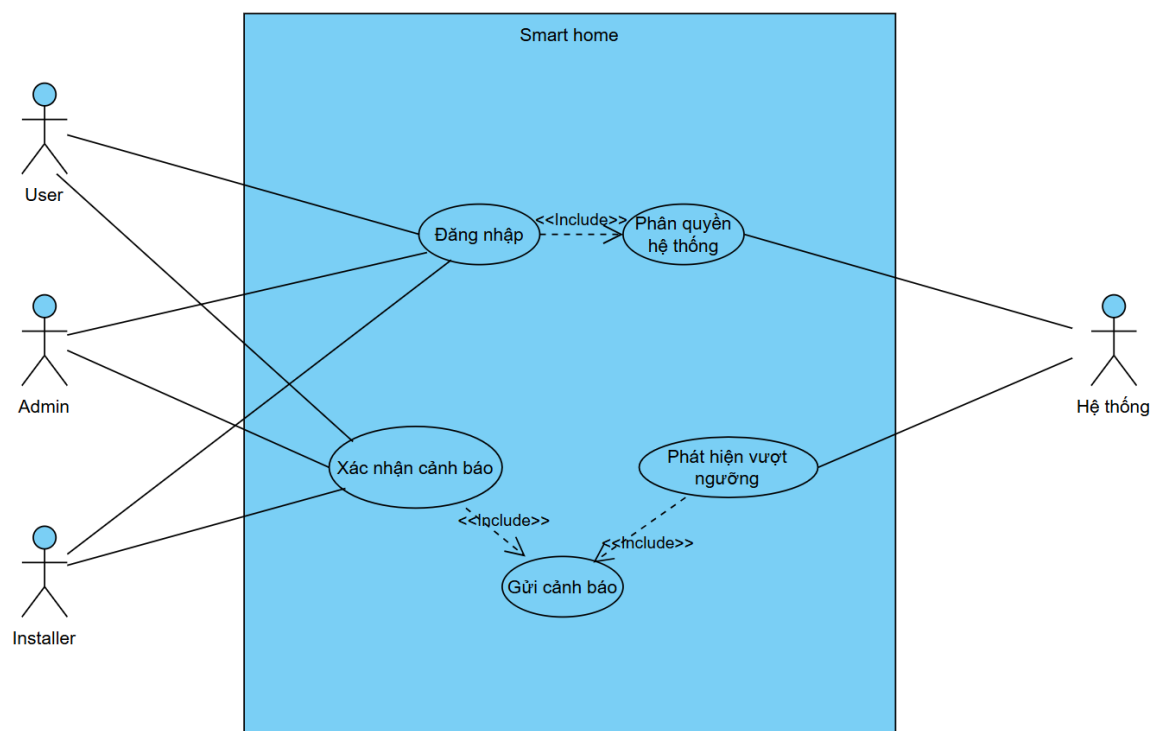
Usecase name	Xem và lọc log hệ thống
Description	Admin xem và lọc log theo tiêu chí.
Actors	Admin
Preconditions	Đã có dữ liệu log.
Trigger	Truy cập trang log.
Post-conditions	Hiển thị danh sách log phù hợp điều kiện lọc.
Main flow	- Chọn bộ lọc (thời gian, loại sự kiện, người dùng). - Hệ thống truy vấn database. - Hiển thị kết quả.
Alternative flow	Không có kết quả → Hiển thị rỗng.
Exceptions	Lỗi truy vấn database.

USE CASE 18: PHÂN QUYỀN HỆ THỐNG

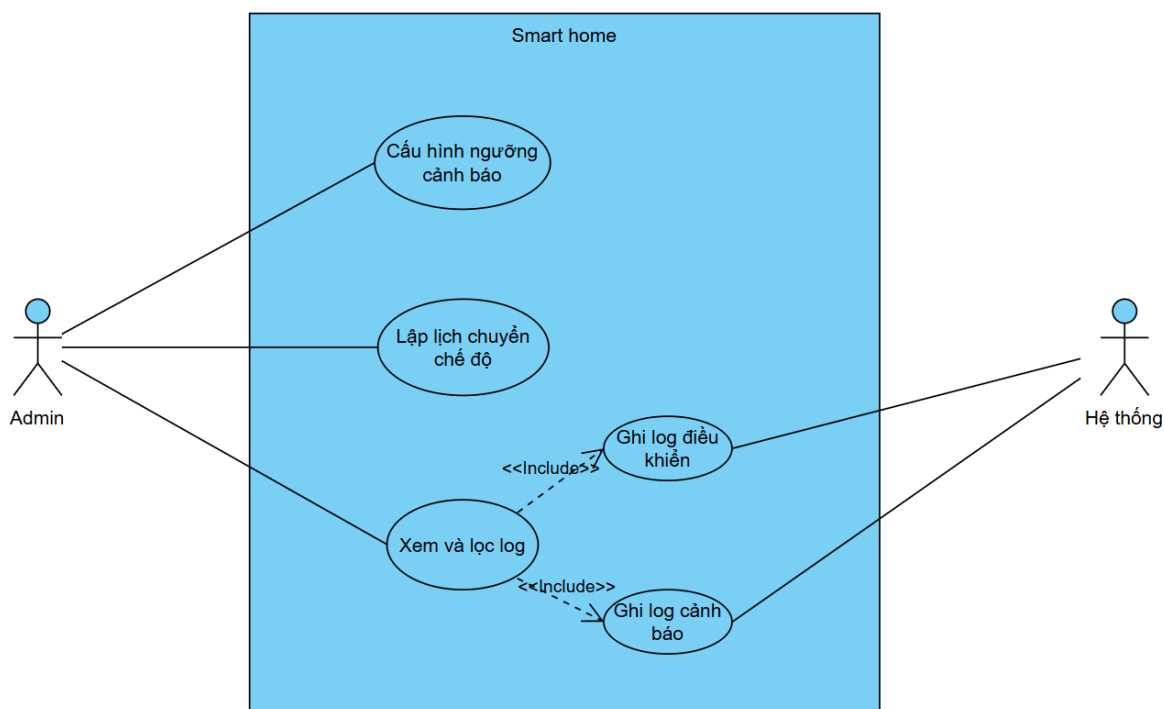
Usecase name	Phân quyền truy cập
Description	Hệ thống phân quyền theo role (Admin, User, Installer).
Actors	System
Preconditions	Người dùng đã đăng nhập.
Trigger	Sau khi xác thực thành công.
Post-conditions	Người dùng chỉ truy cập chức năng phù hợp role.
Main flow	- Xác định role người dùng. - Áp dụng quyền truy cập tương ứng. - Chặn truy cập trái phép.
Alternative flow	Role không hợp lệ → Từ chối truy cập.
Exceptions	Lỗi kiểm tra quyền.

USE CASE 19: DASHBOARD THEO TỪNG CHỦ NHÀ

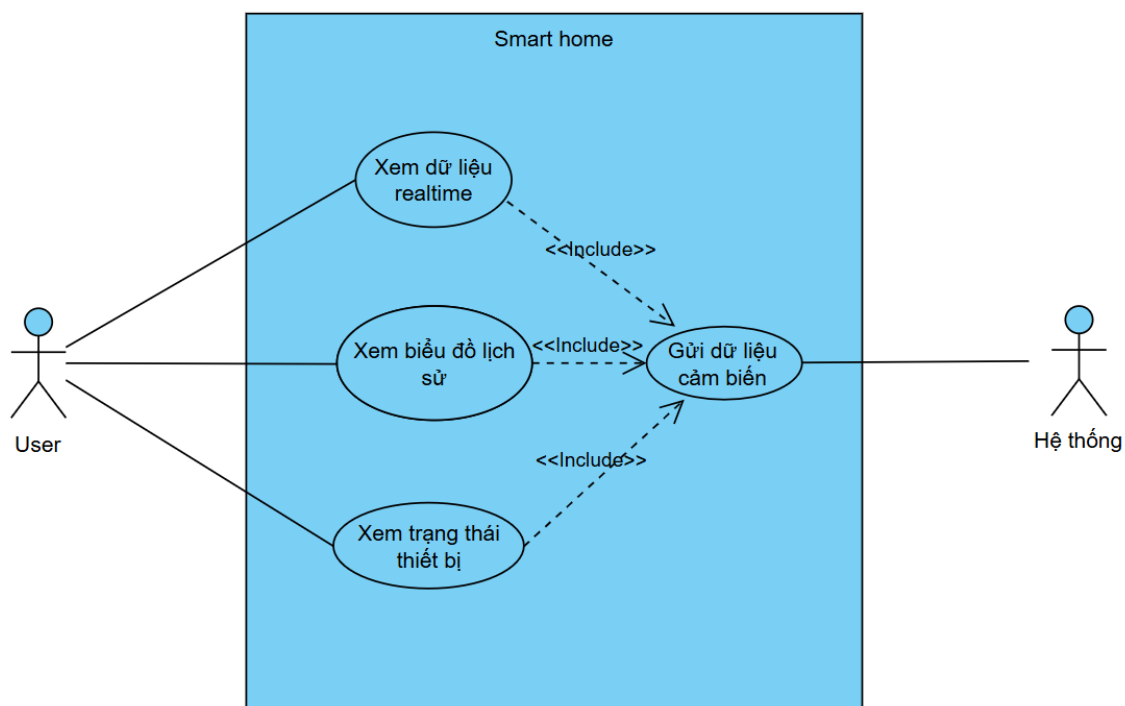
Usecase name	Dashboard riêng theo từng chủ nhà
Description	Mỗi User chỉ xem và điều khiển thiết bị thuộc home của mình.
Actors	User
Preconditions	User đã đăng nhập.
Trigger	Truy cập dashboard.
Post-conditions	Chỉ hiển thị thiết bị thuộc quyền sở hữu.
Main flow	1. Lấy home_id của user. 2. Truy vấn danh sách thiết bị tương ứng. 3. Hiển thị dashboard cá nhân hóa.
Alternative flow	Không có thiết bị → Hiển thị dashboard trống.
Exceptions	Lỗi truy vấn database.



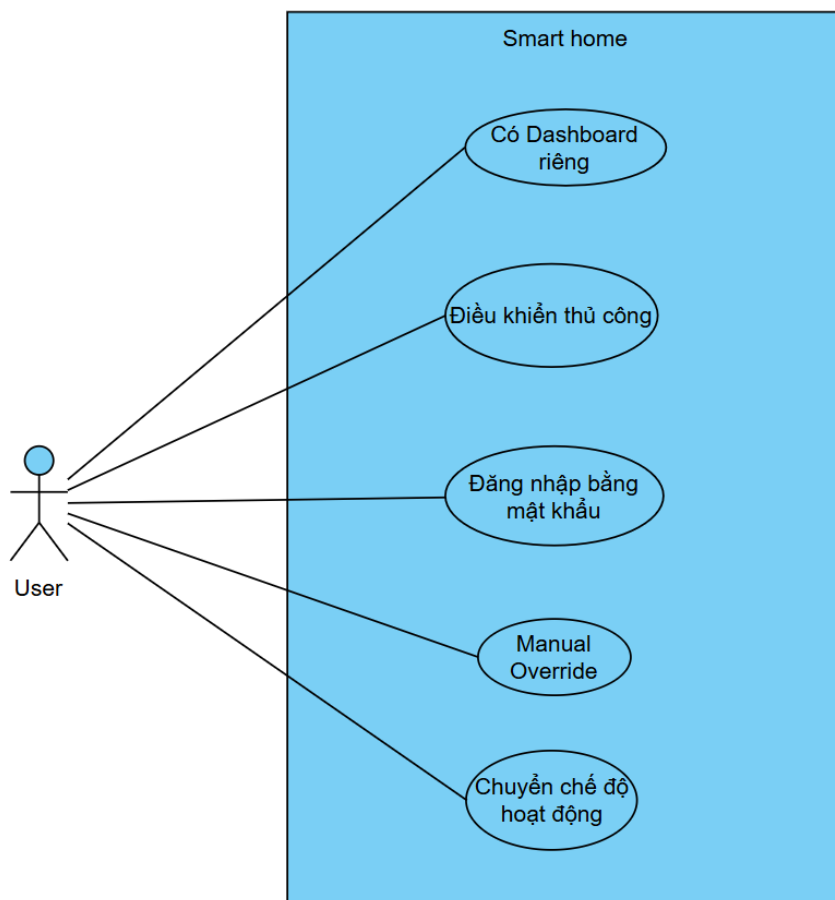
Hình 1: Biểu đồ use-case 1,7,8,9,18



Hình 2: Biểu đồ use-case 6,14,15,16,17



Hình 3: Biểu đồ use-case 2,3,4,5



Hình 4: Biểu đồ use-case 10,11,12,13

2.3 Yêu cầu chức năng

Danh sách các actor:

- User (Chủ nhà)
- Admin (Quản trị hệ thống)
- Installer (Bên lắp đặt)
- IoT Device (Thiết bị cảm biến & điều khiển)
- System (Hệ thống tự động)

Để hiểu rõ thêm về yêu cầu chức năng của hệ thống nhóm tác giả đã mô hình hóa bằng các sơ đồ use-case sau:

USE CASE 1: ĐĂNG NHẬP HỆ THỐNG

Usecase name	Đăng nhập hệ thống
Description	Người dùng đăng nhập vào hệ thống để truy cập chức năng theo role.
Actors	User / Admin / Installer
Preconditions	Tài khoản hợp lệ tồn tại trong hệ thống.
Trigger	Người dùng nhập username và password.
Post-conditions	Success: Tạo session/token và chuyển đến dashboard. Failure: Không tạo session.
Main flow	1. Nhập thông tin đăng nhập. 2. Hệ thống xác thực. 3. Xác định role. 4. Chuyển hướng dashboard.
Alternative flow	Sai thông tin → Thông báo lỗi.
Exceptions	Lỗi server hoặc database.

USE CASE 2: GỬI DỮ LIỆU CẢM BIẾN

Usecase name	Gửi dữ liệu cảm biến
Description	Thiết bị gửi dữ liệu định kỳ về server.
Actors	IoT Device
Preconditions	Thiết bị đang hoạt động và có kết nối mạng.
Trigger	Đến chu kỳ gửi dữ liệu.
Post-conditions	Dữ liệu được lưu vào database.
Main flow	1. Thu thập dữ liệu. 2. Gửi lên server. 3. Server kiểm tra hợp lệ. 4. Lưu database.
Alternative flow	Dữ liệu không hợp lệ → Bỏ qua và ghi log.
Exceptions	Mất kết nối mạng.

USE CASE 3: XEM DỮ LIỆU REALTIME

Usecase name	Xem dữ liệu realtime
Description	Người dùng xem dữ liệu cập nhật theo thời gian thực.
Actors	User
Preconditions	Đã đăng nhập thành công.
Trigger	Mở dashboard.
Post-conditions	Dữ liệu hiển thị ≤ 2 giây sau khi cập nhật.
Main flow	1. Truy cập dashboard. 2. Hệ thống lấy dữ liệu mới nhất. 3. Hiển thị lên giao diện.
Alternative flow	Không có dữ liệu $\rightarrow$ Hiển thị trạng thái chờ.
Exceptions	Lỗi truy vấn database.

USE CASE 4: XEM BIỂU ĐỒ LỊCH SỬ

Usecase name	Xem biểu đồ lịch sử
Description	Xem dữ liệu theo 1h, 24h, 7 ngày.
Actors	User
Preconditions	Có dữ liệu lưu trữ.
Trigger	Chọn khoảng thời gian.
Post-conditions	Hiển thị biểu đồ tương ứng.
Main flow	1. Chọn loại dữ liệu. 2. Chọn mốc thời gian. 3. Hệ thống truy vấn. 4. Hiển thị biểu đồ.
Alternative flow	Không có dữ liệu $\rightarrow$ Thông báo.
Exceptions	Lỗi truy vấn.

USE CASE 5: XEM TRẠNG THÁI THIẾT BỊ

Usecase name	Xem trạng thái thiết bị
Description	Hiển thị ON/OFF hoặc Offline.
Actors	User
Preconditions	Đã đăng nhập.
Trigger	Mở dashboard.
Post-conditions	Trạng thái hiển thị chính xác.
Main flow	1. Hệ thống lấy trạng thái thiết bị. 2. Hiển thị ON/OFF.
Alternative flow	Không phản hồi $\rightarrow$ Hiển thị Offline.
Exceptions	Mất kết nối thiết bị.

USE CASE 6: CẤU HÌNH NGUỒN CẢNH BÁO

Usecase name	Cấu hình ngưỡng cảnh báo
Description	Admin thiết lập giá trị ngưỡng high/low/critical cho cảm biến.
Actors	Admin
Preconditions	Admin đã đăng nhập.
Trigger	Admin truy cập trang cấu hình.
Post-conditions	Ngưỡng mới được lưu vào hệ thống.
Main flow	- Mở trang cấu hình. - Nhập giá trị ngưỡng. - Nhấn lưu. - Hệ thống kiểm tra hợp lệ. - Lưu vào database.
Alternative flow	Giá trị không hợp lệ → Thông báo lỗi.
Exceptions	Lỗi lưu database.

USE CASE 7: PHÁT HIỆN VƯỢT NGUỒN

Usecase name	Phát hiện vượt ngưỡng
Description	Hệ thống kiểm tra dữ liệu và phát hiện giá trị bất thường.
Actors	System
Preconditions	Có dữ liệu cảm biến mới.
Trigger	Nhận dữ liệu từ thiết bị.
Post-conditions	Tạo cảnh báo trạng thái Active nếu vượt ngưỡng.
Main flow	- Nhận dữ liệu mới. - So sánh với ngưỡng cấu hình. - Kiểm tra thời gian duy trì vượt ngưỡng. - Tạo cảnh báo.
Alternative flow	Không vượt ngưỡng → Không tạo cảnh báo.
Exceptions	Lỗi xử lý dữ liệu.

USE CASE 8: GỬI CẢNH BÁO

Usecase name	Gửi cảnh báo
Description	Hệ thống gửi thông báo khi có cảnh báo mới.
Actors	System
Preconditions	Cảnh báo đã được tạo.
Trigger	Trạng thái cảnh báo = Active.
Post-conditions	Người dùng và installer nhận thông báo.
Main flow	- Hiển thị cảnh báo trên dashboard. - Gửi notification/email. - Ghi log sự kiện.
Alternative flow	Gửi email thất bại → Ghi log lỗi.
Exceptions	Lỗi dịch vụ gửi email.

USE CASE 9: XÁC NHẬN CẢNH BÁO

Usecase name	Xác nhận cảnh báo
Description	User/Admin xác nhận cảnh báo đang Active và Installer xác nhận Resolved sau khi giải quyết.
Actors	User / Admin / Installer
Preconditions	Có cảnh báo trạng thái Active.
Trigger	Admin hoặc User nhấn nút Acknowledge. Installer nhấn nút Resolved
Post-conditions	Trạng thái chuyển sang Acknowledged khi xác nhận lỗi và chuyển sang Resolved khi đã sửa.
Main flow	1. Chọn cảnh báo. 2. Nhấn Acknowledge/Resolved. 3. Hệ thống cập nhật trạng thái. 4. Ghi log.
Alternative flow	Cảnh báo đã được xử lý trước đó.
Exceptions	Lỗi cập nhật database.

USE CASE 10: ĐIỀU KHIỂN THỦ CÔNG

Usecase name	Điều khiển thiết bị thủ công
Description	User bật/tắt thiết bị từ dashboard.
Actors	User
Preconditions	Thiết bị đang online.
Trigger	Nhấn nút ON/OFF.
Post-conditions	Thiết bị đổi trạng thái thành công.
Main flow	1. User chọn thiết bị. 2. Nhấn ON/OFF. 3. Hệ thống gửi lệnh. 4. Thiết bị phản hồi. 5. Cập nhật giao diện.
Alternative flow	Thiết bị không phản hồi → Thông báo lỗi.
Exceptions	Mất kết nối HTTP/API hoặc thiết bị.

USE CASE 11: XÁC THỰC MẬT KHẨU NHÀ

Usecase name	Xác thực mật khẩu nhà
Description	Kiểm tra quyền truy cập qua mật khẩu tại thiết bị.
Actors	User
Preconditions	Thiết bị hoạt động.
Trigger	Người dùng nhập mật khẩu.
Post-conditions	Đăng nhập thành công.
Main flow	1. Người dùng nhập mật khẩu 2. Hệ thống kiểm tra mật khẩu chính xác. 3. Nếu đúng, cho phép truy cập hệ thống
Alternative flow	Nhập mật khẩu sai → Hệ thống đếm số lần nhập sai → Nếu quá 3 lần sai, hệ thống báo động, gửi cảnh báo cho chủ nhà, ghi log cho sự kiện
Exceptions	Mất kết nối hệ thống hoặc thiết bị.

USE CASE 12: MANUAL OVERRIDE

Usecase name	Manual Override
Description	Chuyển sang chế độ MANUAL khi user điều khiển trong AUTO.
Actors	User
Preconditions	Hệ thống đang ở chế độ AUTO.
Trigger	User thực hiện điều khiển thủ công.
Post-conditions	Sau Thold phút quay lại AUTO.
Main flow	1. Phát hiện lệnh thủ công. 2. Chuyển sang MANUAL. 3. Giữ trạng thái Thold phút. 4. Tự động quay lại AUTO.
Alternative flow	User tiếp tục thao tác → Reset thời gian Thold.
Exceptions	Lỗi timer hệ thống.

USE CASE 13: CHUYỂN CHẾ ĐỘ HOẠT ĐỘNG

Usecase name	Chuyển chế độ hoạt động
Description	Người dùng chuyển đổi giữa AUTO, SLEEP, AWAY.
Actors	User
Preconditions	Đã đăng nhập hệ thống.
Trigger	Chọn chế độ trên dashboard.
Post-conditions	Hệ thống áp dụng rule tương ứng với mode mới.
Main flow	1. Người dùng chọn mode. 2. Hệ thống xác nhận yêu cầu. 3. Cập nhật mode hiện tại. 4. Áp dụng rule điều khiển thiết bị.
Alternative flow	Mode đã được chọn trước đó → Không thay đổi.
Exceptions	Lỗi cập nhật database.

USE CASE 14: LẬP LỊCH CHUYỂN CHẾ ĐỘ

Usecase name	Lập lịch chuyển chế độ
Description	Admin cấu hình lịch tự động chuyển mode theo thời gian.
Actors	Admin
Preconditions	Admin đã đăng nhập.
Trigger	Truy cập trang cấu hình lịch.
Post-conditions	Lịch được lưu và tự động thực thi theo thời gian cấu hình.
Main flow	1. Nhập thời gian bắt đầu/kết thúc. 2. Chọn mode tương ứng. 3. Lưu cấu hình. 4. Hệ thống kích hoạt scheduler.
Alternative flow	Thời gian trùng lặp → Thông báo lỗi.
Exceptions	Lỗi hệ thống scheduler.

USE CASE 15: GHI LOG ĐIỀU KHIỂN

Usecase name	Ghi log điều khiển thiết bị
Description	Hệ thống ghi nhận mọi hành động bật/tắt thiết bị.
Actors	System
Preconditions	Có hành động điều khiển thiết bị.
Trigger	User/Admin thực hiện điều khiển.
Post-conditions	Log được lưu vào database.
Main flow	1. Nhận sự kiện điều khiển. 2. Ghi user, thời gian, trạng thái trước/sau. 3. Lưu database.
Alternative flow	Không có thay đổi trạng thái → Không ghi log.
Exceptions	Lỗi ghi database.

USE CASE 16: GHI LOG CẢNH BÁO

Usecase name	Ghi log cảnh báo
Description	Lưu lại toàn bộ vòng đời của cảnh báo.
Actors	System
Preconditions	Có cảnh báo được tạo.
Trigger	Trạng thái cảnh báo thay đổi.
Post-conditions	Lưu thời điểm Active/Acknowledge/Resolved.
Main flow	1. Nhận sự kiện thay đổi trạng thái. 2. Cập nhật thời điểm tương ứng. 3. Lưu vào database.
Alternative flow	Không thay đổi trạng thái → Không cập nhật.
Exceptions	Lỗi database.

USE CASE 17: XEM & LỌC LOG

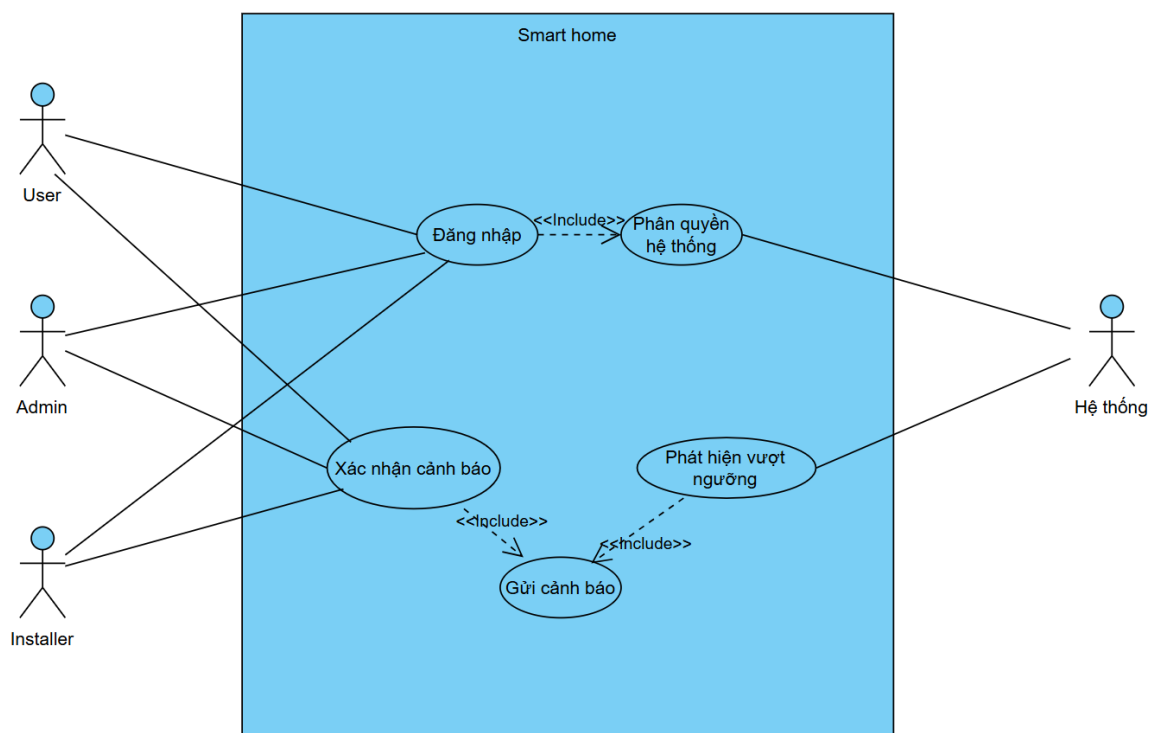
Usecase name	Xem và lọc log hệ thống
Description	Admin xem và lọc log theo tiêu chí.
Actors	Admin
Preconditions	Đã có dữ liệu log.
Trigger	Truy cập trang log.
Post-conditions	Hiển thị danh sách log phù hợp điều kiện lọc.
Main flow	1. Chọn bộ lọc (thời gian, loại sự kiện, người dùng). 2. Hệ thống truy vấn database. 3. Hiển thị kết quả.
Alternative flow	Không có kết quả → Hiển thị rỗng.
Exceptions	Lỗi truy vấn database.

USE CASE 18: PHÂN QUYỀN HỆ THỐNG

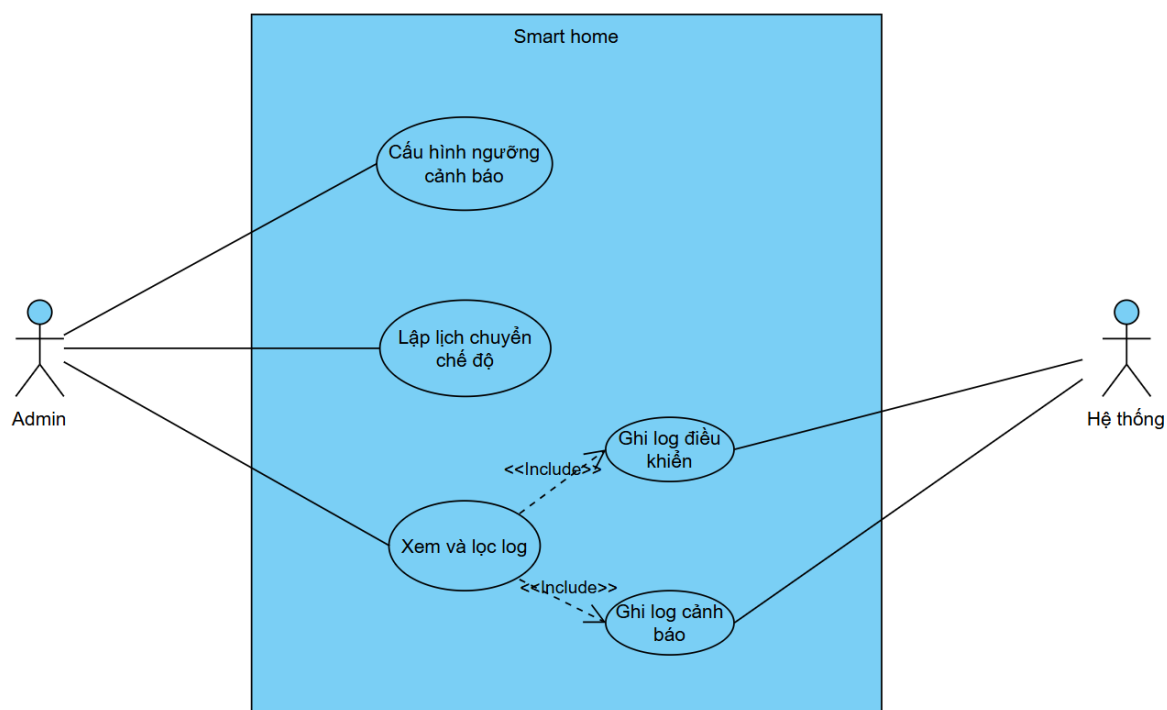
Usecase name	Phân quyền truy cập
Description	Hệ thống phân quyền theo role (Admin, User, Installer).
Actors	System
Preconditions	Người dùng đã đăng nhập.
Trigger	Sau khi xác thực thành công.
Post-conditions	Người dùng chỉ truy cập chức năng phù hợp role.
Main flow	1. Xác định role người dùng. 2. Áp dụng quyền truy cập tương ứng. 3. Chặn truy cập trái phép.
Alternative flow	Role không hợp lệ → Từ chối truy cập.
Exceptions	Lỗi kiểm tra quyền.

USE CASE 19: DASHBOARD THEO TỪNG CHỦ NHÀ

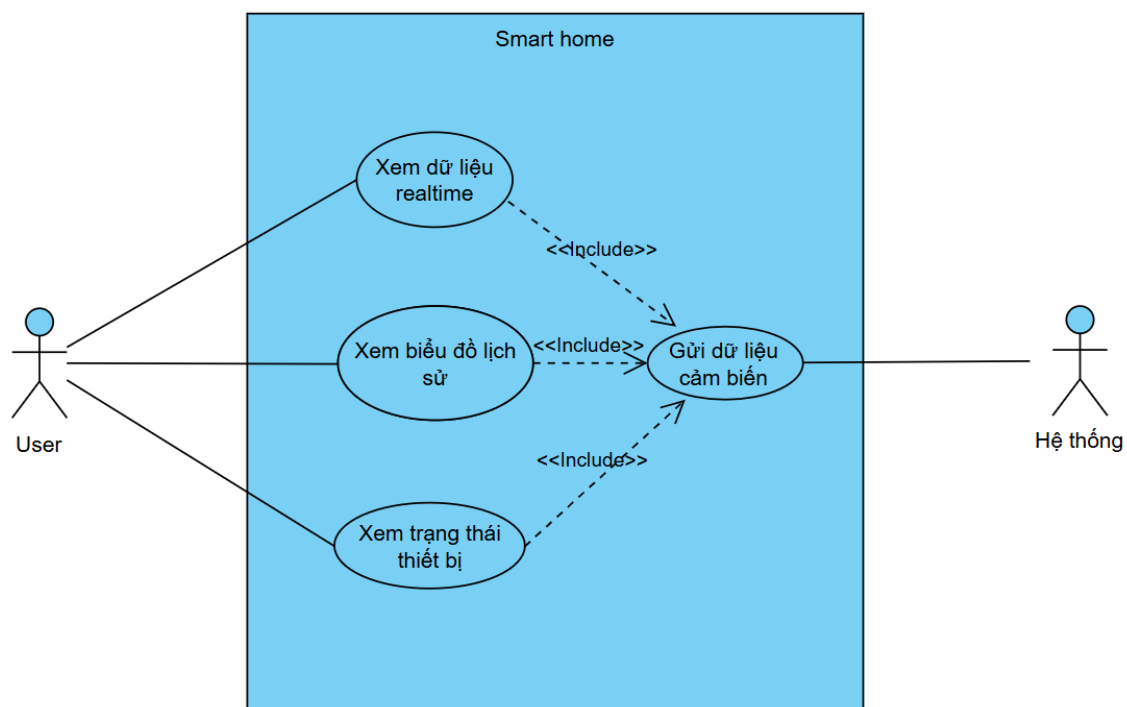
Usecase name	Dashboard riêng theo từng chủ nhà
Description	Mỗi User chỉ xem và điều khiển thiết bị thuộc home của mình.
Actors	User
Preconditions	User đã đăng nhập.
Trigger	Truy cập dashboard.
Post-conditions	Chỉ hiển thị thiết bị thuộc quyền sở hữu.
Main flow	1. Lấy home_id của user. 2. Truy vấn danh sách thiết bị tương ứng. 3. Hiển thị dashboard cá nhân hóa.
Alternative flow	Không có thiết bị → Hiển thị dashboard trống.
Exceptions	Lỗi truy vấn database.



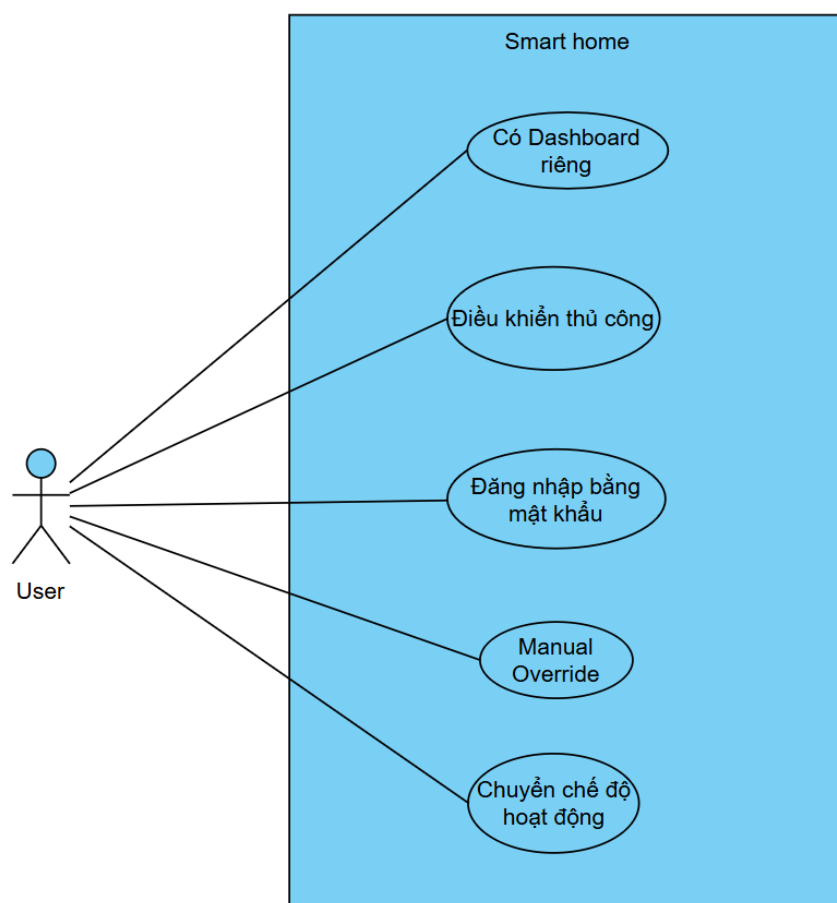
Hình 5: Biểu đồ use-case 1,7,8,9,18



Hình 6: Biểu đồ use-case 6,14,15,16,17



Hình 7: Biểu đồ use-case 2,3,4,5

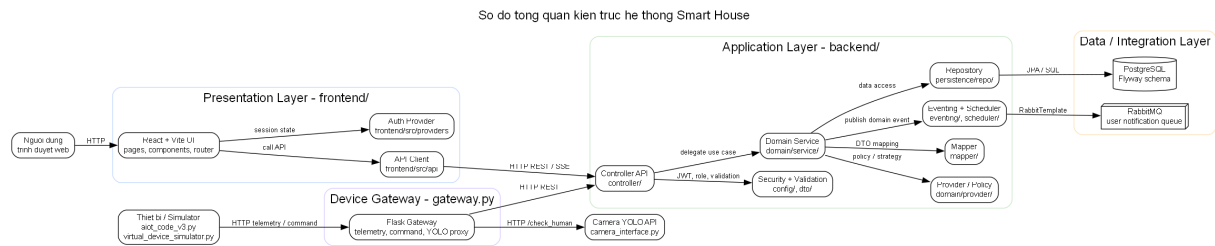


Hình 8: Biểu đồ use-case 10,11,12,13

3 Thiết kế hệ thống và Công nghệ

3.1 Tổng quan kiến trúc hệ thống

Hệ thống Smart House được triển khai theo mô hình web application kết hợp IoT gateway. Frontend React/Vite cung cấp giao diện sử dụng, backend Spring Boot xử lý API và nghiệp vụ, PostgreSQL lưu dữ liệu, RabbitMQ phục vụ luồng thông điệp người dùng, còn gateway Python kết nối thiết bị/simulator với backend và dịch vụ camera YOLO.



Hình 9: Sơ đồ tổng quan kiến trúc hệ thống

Các khối chính trong kiến trúc:

- **Presentation layer:** nằm trong frontend/src, gồm router, pages, components, provider và API client. Tầng này hiển thị dữ liệu, nhận thao tác người dùng và gọi backend bằng HTTP REST hoặc SSE.
- **Application/API layer:** nằm trong backend/src/main/java/com/java/controller, nhận request, kiểm tra dữ liệu đầu vào và trả response chuẩn qua ApiResponse.
- **Business/service layer:** nằm trong domain/service và domain/provider, xử lý điều khiển thiết bị, telemetry, cấu hình, cảnh báo, lịch, audit, xác thực và tự động hóa.
- **Data access layer:** nằm trong persistence/entity và persistence/repo, ánh xạ JPA entity và truy cập PostgreSQL thông qua repository.
- **Integration layer:** gồm gateway.py, camera_interface.py, adapter, eventing, scheduler và RabbitMQ.

Cách giao tiếp giữa các thành phần:

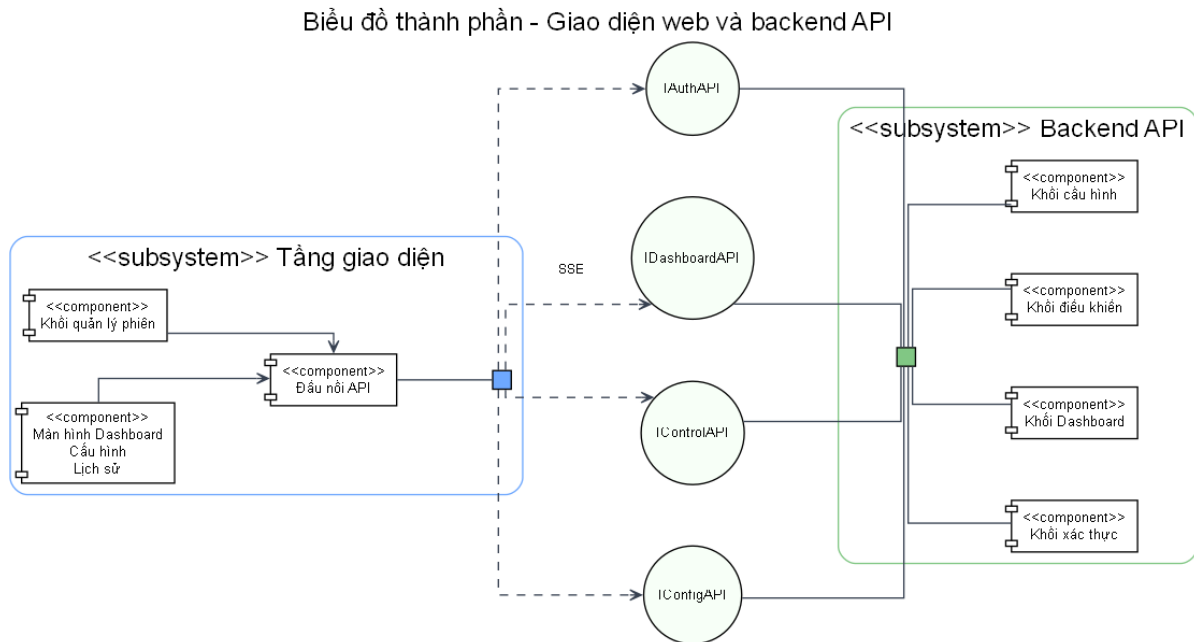
- Trình duyệt người dùng truy cập frontend tại cổng 5173 khi chạy bằng Docker Compose.
- Frontend gọi backend qua VITE_API_BASE_URL; giá trị mặc định trong apiClient.js là http://localhost:8080.
- Backend kết nối PostgreSQL bằng JDBC thông qua các biến DB_URL, DB_USERNAME, DB_PASSWORD.
- Thiết bị hoặc simulator gửi telemetry tới gateway; gateway chuyển tiếp đến endpoint /api/device-telemetry.
- Thiết bị lấy lệnh điều khiển qua endpoint /api/v1/device/{deviceKey}/commands/next và xác nhận bằng /commands/ack.
- Dashboard nhận realtime event qua SSE của DashboardController.

3.2 Mô hình hóa hệ thống

Sơ đồ use case tổng quan: Bốn biểu đồ use case trong phần yêu cầu chức năng mô tả các nhóm nghiệp vụ chính của hệ thống, gồm đăng nhập và phân quyền, cấu hình tự động hóa, giám sát telemetry, điều khiển thiết bị, cảnh báo và truy vết hoạt động.

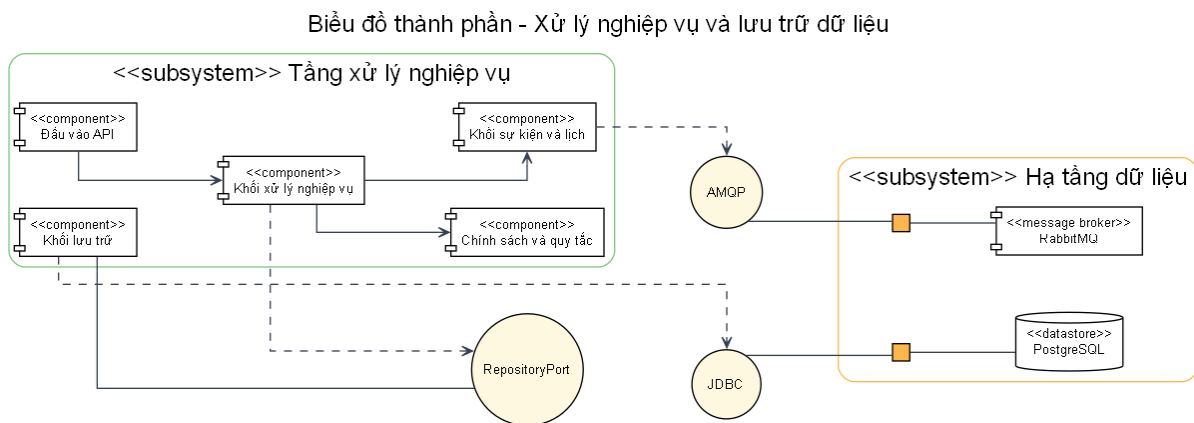
Biểu đồ thành phần:

Hướng dẫn đọc sơ đồ: Các biểu đồ thành phần thể hiện cách các khối chức năng phối hợp trong hệ thống Smart Home. Component biểu diễn một khối chức năng của hệ thống; provided interface thể hiện năng lực mà component cung cấp; required interface thể hiện năng lực mà component cần từ component khác; port là điểm giao tiếp của component; đường nối thể hiện quan hệ phụ thuộc hoặc giao tiếp giữa các khối.



Hình 10: Biểu đồ thành phần cho giao diện web và backend API

Biểu đồ thành phần giao diện web và backend API mô tả ranh giới giao tiếp giữa tầng giao diện và các năng lực phía máy chủ. Tầng giao diện tiếp nhận thao tác của người dùng, quản lý phiên đăng nhập và gửi yêu cầu đến các khối xác thực, Dashboard, điều khiển thiết bị và cấu hình tự động hóa.



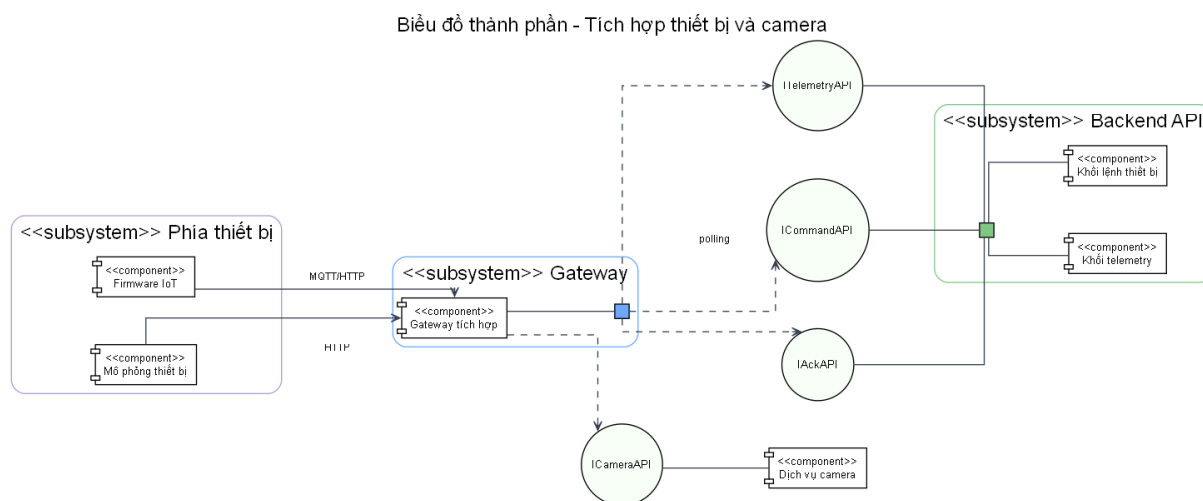
Hình 11: Biểu đồ thành phần cho tầng xử lý nghiệp vụ và lưu trữ dữ liệu

Biểu đồ thành phần tầng xử lý nghiệp vụ và lưu trữ dữ liệu mô tả các khối đảm nhận kiểm tra yêu cầu, áp dụng quy tắc nghiệp vụ, lưu trữ trạng thái hệ thống và phát sinh sự kiện. PostgreSQL lưu dữ liệu bền vững, còn RabbitMQ hỗ trợ các thông điệp bất đồng bộ trong các luồng cần tách rời thời điểm xử lý.

Biểu đồ thành phần tích hợp thiết bị và camera thể hiện gateway là điểm trung gian giữa thiết bị, mô phỏng thiết bị, backend API và dịch vụ camera. Khối này chuyển dữ liệu telemetry, nhận lệnh điều khiển, gửi xác nhận thực thi và gọi dịch vụ nhận diện khi cần thông tin từ camera.

Biểu đồ hoạt động cho luồng đăng nhập:

Biểu đồ hoạt động cho luồng đăng nhập mô tả quá trình người dùng mở màn hình đăng nhập, nhập thông tin hoặc chọn Google, sau đó giao diện web gửi yêu cầu xác thực đến backend. Nếu thông tin không hợp lệ, hệ thống hiển thị thông báo lỗi và kết thúc luồng thất bại. Nếu hợp lệ, backend tạo JWT và thông tin phiên, giao diện lưu phiên và chuyển người dùng đến Dashboard.



Hình 12: Biểu đồ thành phần cho tích hợp thiết bị và camera

Biểu đồ hoạt động cho luồng telemetry và tự động hóa:

Luồng telemetry bắt đầu khi thiết bị hoặc bộ mô phỏng gửi dữ liệu cảm biến về hệ thống. Backend kiểm tra tính hợp lệ của thiết bị và dữ liệu, ghi nhận dữ liệu cảm biến, cập nhật trạng thái thiết bị, phát sự kiện nội bộ và đánh giá các quy tắc tự động hóa đang áp dụng cho ngôi nhà.

Biểu đồ lớp tổng quan:

Biểu đồ lớp mô tả các thực thể nghiệp vụ chính và quan hệ giữa chúng trong phạm vi hệ thống. Người dùng tham gia vào một hoặc nhiều Home thông qua HomeMember và Role. Home bao gồm các thiết bị, cảm biến, lịch chế độ và quy tắc tự động hóa. Dữ liệu cảm biến, trạng thái thiết bị, lệnh điều khiển, cảnh báo và nhật ký hoạt động tạo thành chuỗi thông tin phục vụ giám sát, điều khiển và truy vết trong hệ thống.

Sequence diagram cho luồng gửi telemetry:

Luồng gửi telemetry có actor thiết bị hoặc simulator, gateway, controller, service, database và event bus. Message chính đi từ `gateway.py` đến `TelemetryController`, sau đó `TelemetryIngestService` kiểm tra device/sensor, lưu `sensor_data`, cập nhật runtime state và publish event nội bộ.

Sequence diagram cho luồng thiết bị polling lệnh điều khiển:

Luồng polling tách riêng vòng lặp lấy lệnh để hình không bị rối. `CommandController` gọi `ControlCommandService` để tìm command đang chờ trong PostgreSQL. Khung alt thể hiện hai nhánh: có lệnh thì trả payload để thiết bị thực thi, không có lệnh thì trả kết quả rỗng.

Sequence diagram cho luồng ACK lệnh điều khiển:

Luồng ACK mô tả thiết bị gửi kết quả thực thi về gateway và backend. `ControlCommandService` cập nhật bảng `control_commands`; khung alt thể hiện nhánh ACK hợp lệ và ACK không hợp lệ.

Deployment diagram:

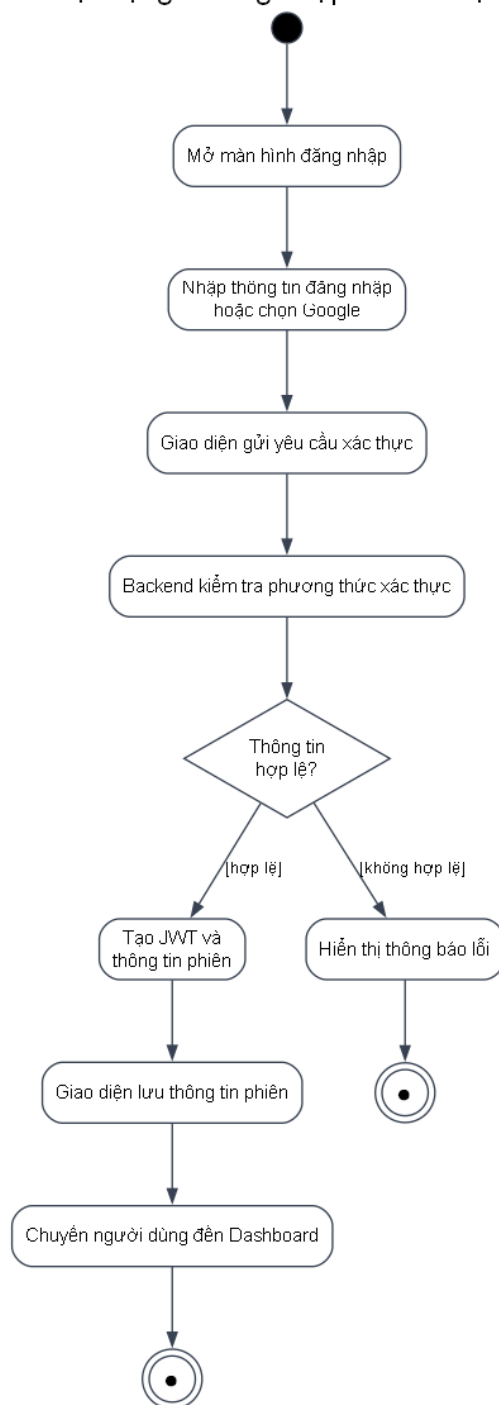
Deployment diagram phản ánh đúng `docker-compose.yml`: service frontend expose 5173:80, service backend expose 8080:8080, service db expose 5432:5432, service rabbitmq expose 5672 và 15672. Volume `postgres_data` lưu dữ liệu PostgreSQL.

3.3 Thiết kế ứng dụng

Thiết kế tầng giao diện:

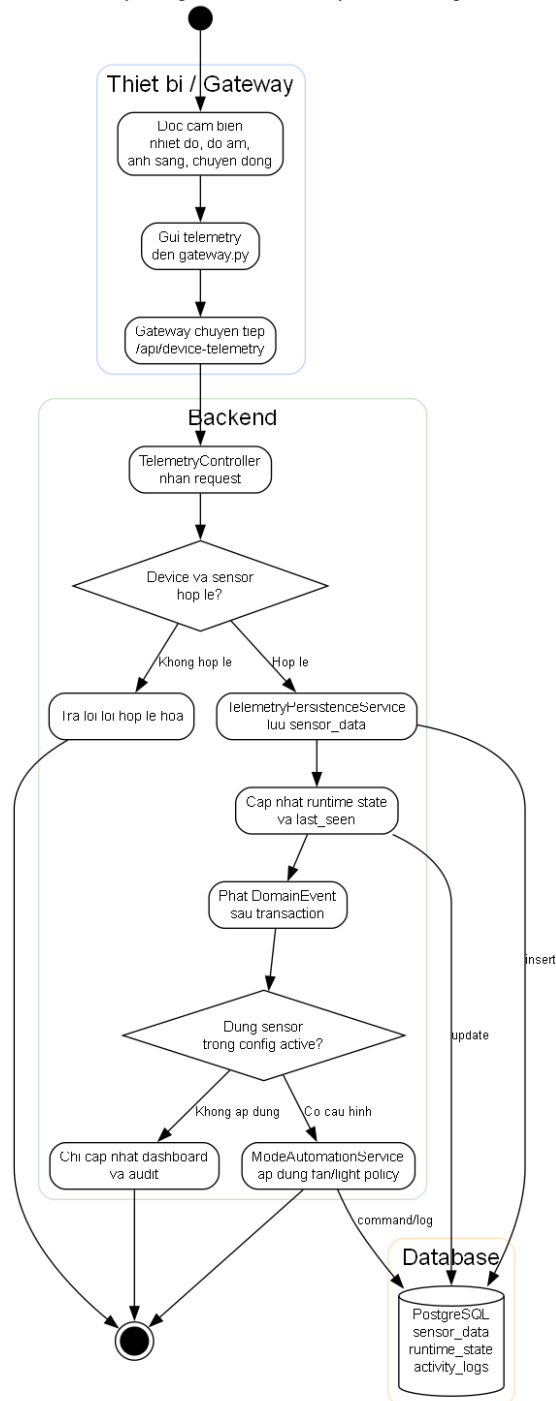
- `frontend/src/router/AppRouter.jsx` định nghĩa các route chính: login, Google callback, dashboard, history, configs, settings, audit logs và trang lỗi.
- `frontend/src/pages` chứa màn hình nghiệp vụ; `frontend/src/components` chứa UI tái sử dụng như thẻ dashboard, bảng lịch, bảng audit, form cấu hình và layout.
- `AuthProvider.jsx`, `ProtectedRoute.jsx`, `RequireHomeRoute.jsx`, `RoleProtectedRoute.jsx` kiểm soát phiên và quyền truy cập trên frontend.

Biểu đồ hoạt động - Đăng nhập và khởi tạo phiên

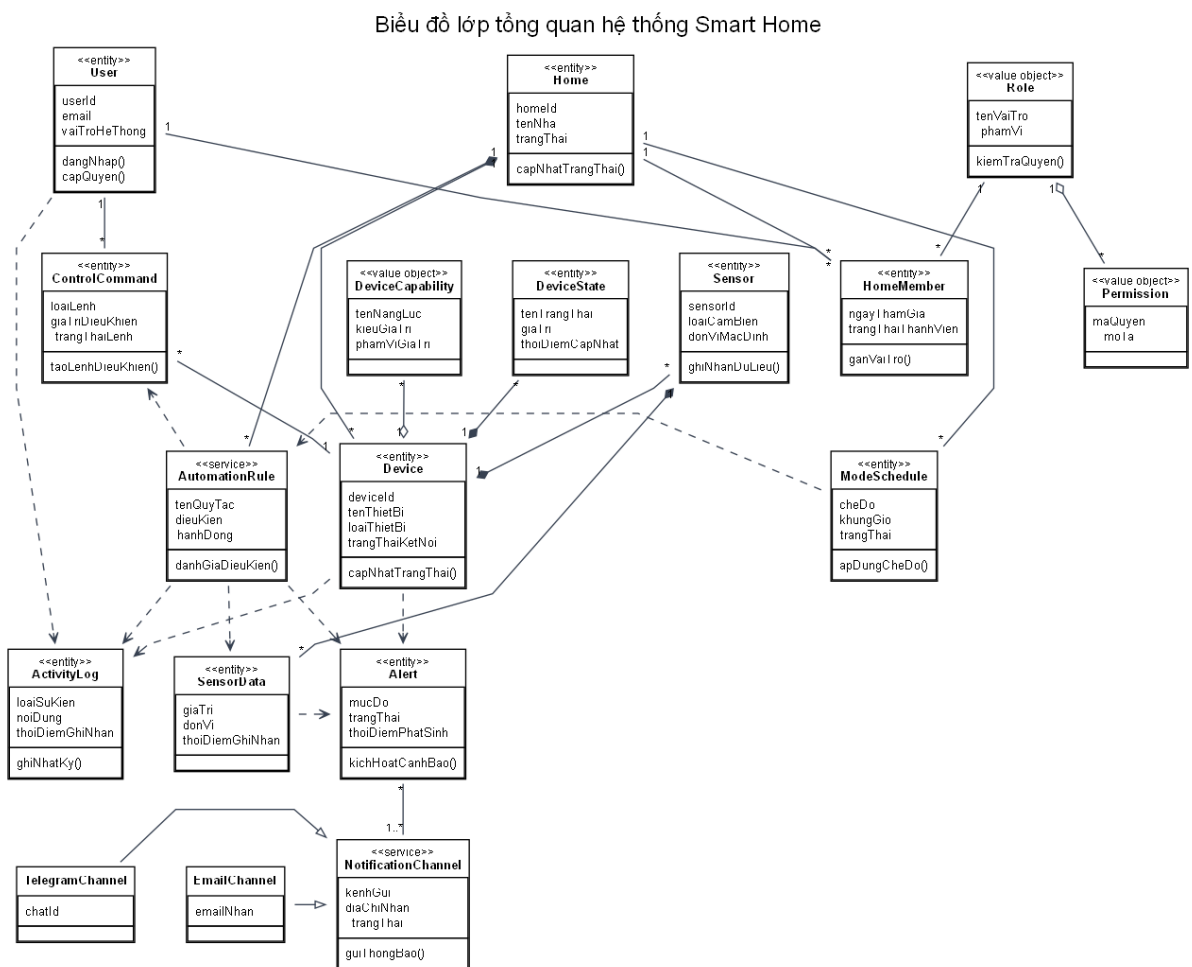


Hình 13: Biểu đồ hoạt động cho luồng đăng nhập và khởi tạo phiên

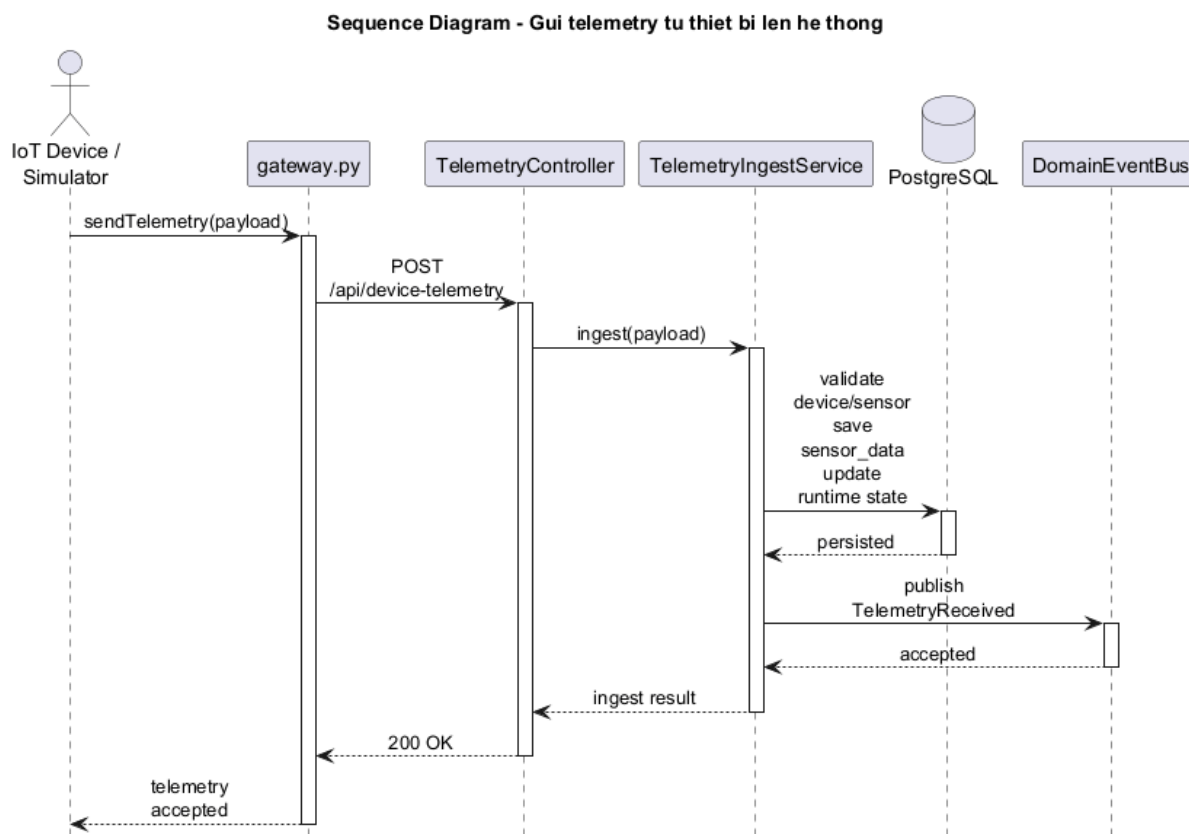
Activity Diagram - Telemetry và tự động hóa



Hình 14: Biểu đồ hoạt động cho luồng telemetry và tự động hóa



Hình 15: Biểu đồ lớp tổng quan hệ thống Smart Home



Hình 16: Sequence diagram cho luồng gửi telemetry

Thiết kế tầng xử lý nghiệp vụ:

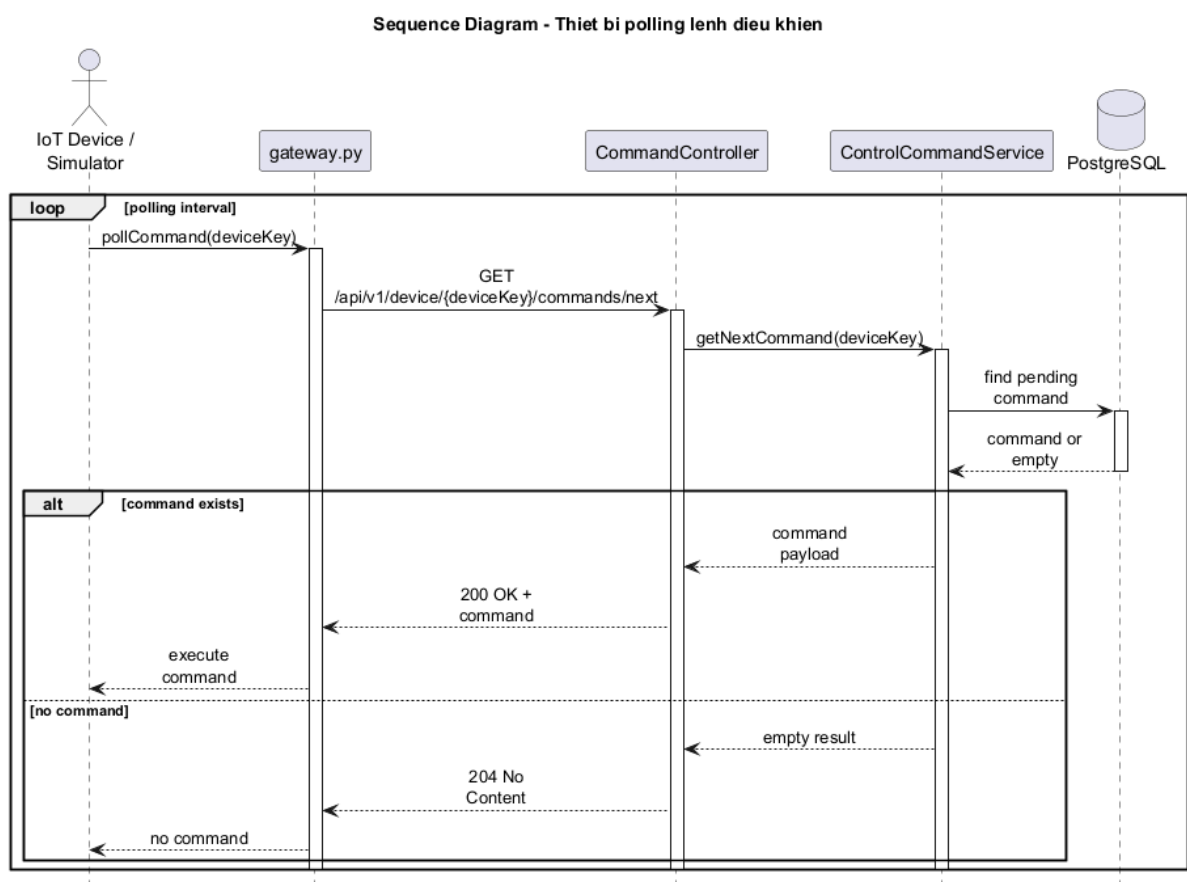
- controller nhận request HTTP và chuyển yêu cầu sang service.
- domain/service điều phối use case như telemetry ingest, control command, config, dashboard, alert, audit, home user, schedule.
- domain/provider chứa strategy, policy và resolver cho xác thực, schedule, runtime state, device target và audit.
- eventing và scheduler xử lý event nội bộ, heartbeat dashboard, offline detection, manual hold, device schedule và outbox publisher.

Thiết kế tầng dữ liệu:

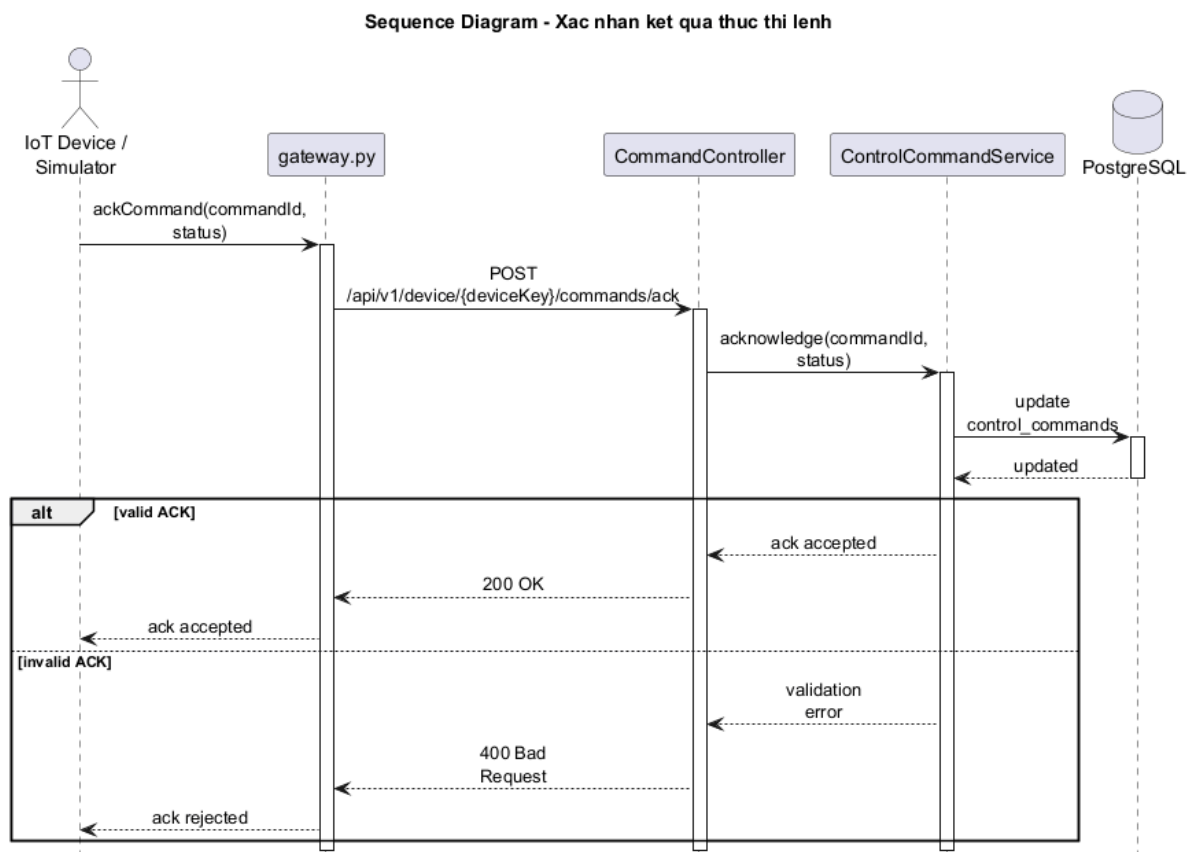
- persistence/entity ánh xạ các bảng PostgreSQL như users, homes, devices, sensors, sensor_data, configs, control_commands, alerts, activity_logs.
- persistence/repo chứa repository kế thừa JpaRepository.
- Migration chính nằm trong backend/src/main/resources/db/migration/V1__init.sql; các migration sau bổ sung cột nguồn lệnh và index phục vụ realtime, audit, dashboard.

Luồng request-response chính:

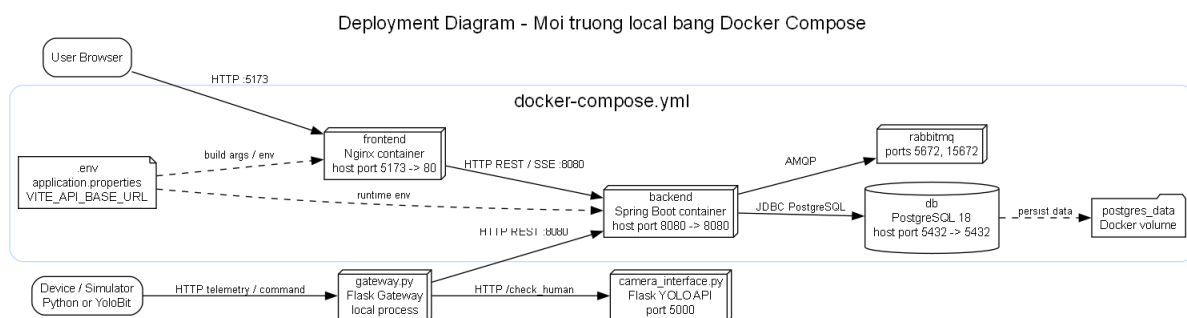
- **Đăng nhập:** LoginPage gọi authApi.js; backend nhận tại AuthController; service xác thực cấp token; frontend lưu phiên trong authStorage.js.
- **Dashboard realtime:** DashboardPage tải summary qua DashboardController; dashboardRealtime.js mở SSE để nhận cập nhật trạng thái.
- **Điều khiển thiết bị:** DeviceSwitchCard và hook dashboard gọi /api/control/devices/{deviceId}; backend đi qua ControlController, ControlFacadeService, ManualControlService, DeviceCommandExecutionService.
- **Telemetry:** thiết bị/gateway gọi /api/device-telemetry; backend lưu dữ liệu qua TelemetryIngestService và TelemetryPersistenceService.
- **Cấu hình tự động hóa:** ConfigPage gọi ConfigController; backend lưu configs, ghi audit và kích hoạt profile active.



Hình 17: Sequence diagram cho luồng thiết bị polling lệnh điều khiển



Hình 18: Sequence diagram cho luồng xác nhận kết quả thực thi lệnh



Hình 19: Deployment diagram cho môi trường chạy local bằng Docker Compose

3.4 Cấu trúc thư mục và vai trò từng thành phần

```
smart-house/
|-- backend/
|   |-- src/main/java/com/java/controller/
|   |-- src/main/java/com/java/domain/service/
|   |-- src/main/java/com/java/domain/provider/
|   |-- src/main/java/com/java/persistence/entity/
|   |-- src/main/java/com/java/persistence/repo/
|   |-- src/main/java/com/java/mapper/
|   |-- src/main/java/com/java/eventing/
|   |-- src/main/java/com/java/scheduler/
|   |-- src/main/resources/db/migration/
|   '-- src/test/java/
|-- frontend/
|   |-- src/pages/
|   |-- src/components/
|   |-- src/api/
|   |-- src/providers/
|   |-- src/router/
|   |-- src/hooks/
|   '-- src/utils/
|-- docs/
|   '-- diagram/
|-- Images/
|-- scripts/
|-- gateway.py
|-- camera_interface.py
|-- virtual_device_simulator.py
|-- aiot_code_v3.py
|-- docker-compose.yml
'-- main.tex, hcmut.sty, Sections/
```

Table 1: Cấu trúc thư mục và vai trò từng thành phần

Thư mục / file	Loại nội dung	Vai trò trong hệ thống	Runtime / build / tài liệu
backend/	Spring Boot project, Maven, Dockerfile	Backend API, nghiệp vụ, persistence, migration, test	Runtime và build
backend/src/main/java/com/java/controller	REST controller	Endpoint cho auth, dashboard, telemetry, control, config, alert, audit, schedule	Runtime
backend/src/main/java/com/java/domain/service	Service nghiệp vụ	Điều phối use case và xử lý logic chính	Runtime
backend/src/main/java/com/java/domain/provider	Strategy, policy, resolver	Tách quy tắc nghiệp vụ và lựa chọn chiến lược	Runtime
backend/src/main/java/com/java/persistence	Entity và repository	Ánh xạ JPA và truy cập database	Runtime
backend/src/main/resources/db/migration	Flyway SQL migration	Khởi tạo schema, seed data, index và ràng buộc	Runtime khi ứng dụng khởi động
frontend/	React/Vite project, Dockerfile, Nginx config	Giao diện web và bản build static	Runtime frontend và build
frontend/src/pages	Route-level screens	Dashboard, History, Config, Settings, Audit, Auth	Runtime frontend
frontend/src/components	UI component	Card, table, modal, layout, form và filter	Runtime frontend
frontend/src/api	API client và service	Gọi backend, gắn token, refresh token, chuẩn hóa lỗi	Runtime frontend
frontend/src/providers	React Context provider	Quản lý trạng thái xác thực	Runtime frontend
docs/diagram	Sơ đồ tài liệu	Tài liệu kiến trúc, activity, sequence, state và sơ đồ sinh mới	Documentation

Thư mục / file	Loại nội dung	Vai trò trong hệ thống	Runtime / build / tài liệu
Images/	Hình report	Logo, use case, ERD, mapping, sơ đồ Graphviz sinh mới	Documentation
gateway.py	Flask gateway	Proxy telemetry, command, ack, YOLO và cache cục bộ cho thiết bị	Runtime hỗ trợ thiết bị
camera_interface.py	Flask + OpenCV + YOLO	API nhận diện người/chuyển động qua camera	Runtime tùy chọn
virtual_device_simulator.py	Python simulator	Mô phỏng telemetry, command, IR và trạng thái thiết bị	Runtime demo
aiot_code_v3.py	MicroPython/YoloBit	Mã thiết bị đọc cảm biến, điều khiển phần cứng và giao tiếp gateway	Runtime thiết bị
scripts/	Test script hỗ trợ	Kiểm thử gateway và cache camera	Test/support
Sections/, main.tex	LaTeX report	Nội dung báo cáo kỹ thuật	Documentation

3.5 Design pattern được sử dụng

Table 2: Design pattern được sử dụng trong hệ thống

Pattern	Vị trí hiện thực hóa	Cách xuất hiện trong ứng dụng	Lợi ích
Layered Architecture	controller, domain/service, persistence/repo, persistence/entity, mapper	Backend chia tầng HTTP, nghiệp vụ, ánh xạ dữ liệu và lưu trữ	Dễ bảo trì, giảm coupling, thuận lợi kiểm thử
MVC / REST Controller	controller package	Controller nhận request, service xử lý, entity/repository là model lưu trữ	Tách giao tiếp HTTP khỏi nghiệp vụ
Service Layer	TelemetryIngestService, ControlFacadeService, ConfigService, DashboardService	Service điều phối use case và transaction	Tập trung logic nghiệp vụ trong một tầng rõ ràng
Repository Pattern	DeviceRepository, ConfigRepository, SensorDataRepository, ControlCommandRepository	Repository kế thừa JpaRepository để truy cập dữ liệu	Che giấu chi tiết truy vấn và ORM khỏi service
Dependency Injection	Các class dùng @Service, @Component, @RequiredArgsConstructor	Spring tiêm dependency qua constructor	Dễ thay implementation và viết test
Strategy Pattern	AuthenticationStrategy, ThresholdRuleStrategy, ScheduleExecutionStrategy	Nhiều implementation cùng hợp đồng, resolver chọn strategy phù hợp	Mở rộng hành vi mà không làm phình service chính
Adapter Pattern	DeviceCommandAdapter, MockOhstemDeviceAdapter	Adapter tách nghiệp vụ điều khiển khỏi chi tiết thiết bị	Dễ thay đổi cách giao tiếp thiết bị
Factory Pattern	ControlCommandFactory, DeviceEntityFactory, DeviceRuntimeStateFactory	Gom quy tắc tạo entity và giá trị mặc định	Giảm lặp logic khởi tạo
Observer-style Eventing	DomainEventBus, DomainEventListener, SimpleDomainEventBus	Service publish event, listener xử lý realtime/audit/alert sau transaction	Giảm phụ thuộc trực tiếp giữa các module
DTO / Mapper Pattern	controller/dto, mapper	Request/response tách khỏi entity database	API ổn định và hạn chế lộ cấu trúc lưu trữ
Route Guard Pattern	ProtectedRoute.jsx, RequireHomeRoute.jsx, RoleProtectedRoute.jsx	Frontend bảo vệ route theo trạng thái đăng nhập và role	Kiểm soát truy cập ở tầng giao diện
Provider Pattern	AuthProvider.jsx	React Context cung cấp auth state cho toàn ứng dụng	Tránh truyền props qua nhiều tầng

3.6 Công nghệ sử dụng và lợi ích

Table 3: Công nghệ sử dụng và lợi ích trong hệ thống

Công nghệ	Vị trí hiện thực hóa	Dùng ở đâu	Lợi ích trong hệ thống
Java / Spring Boot	backend/pom.xml, SmartHouseApplication.java	Backend API và service	Nền tảng ổn định cho REST API, DI, validation, scheduler
Spring Web MVC	spring-boot-starter-webmvc	Controller HTTP, SSE	Triển khai endpoint và real-time stream
Spring Data JPA/JDBC	spring-boot-starter-data-jpa, spring-boot-starter-data-jdbc	Repository và entity	Truy cập PostgreSQL theo ORM/repository
Spring Security + JWT	SecurityConfig.java, JwtService.java, pom.xml	JWT, route protection, password encoder	Xác thực stateless và phân quyền API
Flyway	spring-boot-starter-flyway, db/migration	Migration database	Quản lý schema và seed nhất quán
PostgreSQL	docker-compose.yml, application.properties	Database chính	Lưu dữ liệu quan hệ, ràng buộc, index, JSONB audit
RabbitMQ / Spring AMQP	RabbitMqConfig.java, docker-compose.yml	Outbox event và notification queue	Tách phát thông điệp khỏi transaction database
React	frontend/package.json	Frontend UI	Xây dựng giao diện component-based
Vite	frontend/package.json, vite.config.js	Dev/build frontend	Build nhanh và cấu hình gọn

Công nghệ	Vị trí hiện thực hóa	Dùng ở đâu	Lợi ích trong hệ thống
React Router	frontend/package.json, AppRouter.jsx	Routing và route guard	Điều hướng màn hình và bảo vệ quyền truy cập
Recharts	frontend/package.json, HistoryChartCard.jsx	Biểu đồ lịch sử	Hiển thị dữ liệu sensor theo thời gian
Python Flask	gateway.py, camera_interface.py	Gateway thiết bị và YOLO API	Tạo API nhẹ cho integration thiết bị
OpenCV / Ultralytics YOLO	camera_interface.py, yolov8n.pt, yolov8m.pt	Nhận diện người/chuyển động	Bổ sung dữ liệu camera cho kịch bản an ninh
Docker Compose / Nginx	docker-compose.yml, frontend/Dockerfile, frontend/nginx.conf	Chạy frontend/backend/db/rabbitmq local	Đồng bộ môi trường demo và triển khai local

3.7 Hướng dẫn cài đặt, cấu hình và sử dụng

Yêu cầu môi trường:

- Java JDK dùng để chạy backend. pom.xml khai báo java.version=17; Dockerfile backend dùng image build/run nền Java 21.
- Node.js và npm để cài dependency frontend; Dockerfile frontend dùng node:20-alpine.
- PostgreSQL cho database; Docker Compose dùng postgres:18-alpine.
- RabbitMQ cho message queue; Docker Compose dùng rabbitmq:4.0-management.
- Python để chạy gateway.py, camera_interface.py và virtual_device_simulator.py.
- Docker và Docker Compose khi chạy toàn bộ stack bằng container.

Cấu hình chính:

- Root project có file .env được docker-compose.yml sử dụng cho database, backend, RabbitMQ, mail, Telegram và Google OAuth.
- Backend đọc cấu hình trong backend/src/main/resources/application.properties, gồm server.port=8080, DB_URL, FLYWAY_ENABLED, JWT, RabbitMQ, mail và Telegram.
- Frontend nhận VITE_API_BASE_URL, VITE_GOOGLE_CLIENT_ID, VITE_GOOGLE_REDIRECT_URI qua Docker build args hoặc môi trường Vite.

Cài đặt và chạy thử công:

- Backend: vào thư mục backend, chạy `mvnw.cmd test` để kiểm thử hoặc `mvnw.cmd spring-boot:run` để chạy server.
- Frontend: vào thư mục frontend, chạy `npm install`, sau đó `npm run dev`.
- Database: tạo database PostgreSQL trùng với cấu hình DB_URL; Flyway tự chạy migration khi backend khởi động nếu FLYWAY_ENABLED=true.
- Gateway thiết bị: chạy `python gateway.py` sau khi cấu hình biến môi trường liên quan đến backend và token thiết bị.
- Simulator: chạy `python virtual_device_simulator.py` để mô phỏng telemetry và command.
- Camera API: chạy `python camera_interface.py`; service cung cấp /health và /check_human.

Chạy bằng Docker Compose:

- Từ root project, chạy `docker-compose up -d -build`.
- Frontend truy cập tại `http://localhost:5173`.
- Backend chạy tại `http://localhost:8080`.
- PostgreSQL expose tại `localhost:5432`.
- RabbitMQ expose AMQP tại `localhost:5672` và management UI tại `localhost:15672`.

Khởi tạo dữ liệu:

- Flyway migration V1__init.sql tạo schema, enum, trigger, index và dữ liệu seed.
- Các migration V2 đến V5 bổ sung cột source cho command và index phục vụ real-time/audit/dashboard.
- File backend/db/init.sql là script SQL khởi tạo khác đang có trong project.

3.8 Kịch bản demo thực tế theo nghiệp vụ

Kịch bản 1: Giám sát môi trường phòng khách và phản ứng theo tình huống nóng

- **Bối cảnh:** Phòng khách có sensor nhiệt độ, độ ẩm, ánh sáng, chuyển động; quạt và đèn được điều khiển qua dashboard.
- **Vai trò người dùng:** Chủ nhà hoặc thành viên có quyền truy cập home.
- **Dữ liệu đầu vào:** Telemetry từ `virtual_device_simulator.py` hoặc `aiot_code_v3.py`.
- **Các bước thao tác:** Đăng nhập, mở Dashboard, quan sát monitoring card, đổi mode, điều chỉnh quạt/đèn dựa trên dữ liệu, mở History để đối chiếu dữ liệu thời gian.
- **Kết quả mong đợi:** Dashboard hiển thị dữ liệu realtime; `sensor_data`, `device_runtime_state`, `device_state_history` được cập nhật.
- **Thành phần kỹ thuật:** `DashboardPage.jsx`, `dashboardApi.js`, `DashboardController.java`, `TelemetryIngestService.java`, `TelemetryPersistenceService.java`.

Kịch bản 2: Cấu hình tự động hóa theo ngưỡng nhiệt độ và ánh sáng

- **Bối cảnh:** Chủ nhà muốn quạt tự phản ứng với nhiệt độ và đèn tự phản ứng với ánh sáng.
- **Vai trò người dùng:** Owner, co-owner hoặc admin.
- **Dữ liệu đầu vào:** Ngưỡng `thigh`, `tlow`, `lhigh`, `llow`, `tcritical`; thiết bị giám sát và thiết bị đích.
- **Các bước thao tác:** Mở Config, chỉnh threshold, gán sensor/actuator, lưu cấu hình, kích hoạt profile, quan sát audit log.
- **Kết quả mong đợi:** Bảng configs cập nhật; khi telemetry phù hợp đến, `ModeAutomationService` áp dụng policy cho quạt hoặc đèn.
- **Thành phần kỹ thuật:** `ConfigPage.jsx`, `configApi.js`, `ConfigController.java`, `ConfigService.java`, `FanAutomationPolicy.java`, `LightAutomationPolicy.java`.

Kịch bản 3: Truy vết cảnh báo và hoạt động hệ thống

- **Bối cảnh:** Hệ thống phát sinh cảnh báo do sensor, thiết bị offline hoặc thao tác điều khiển.
- **Vai trò người dùng:** Admin hoặc người có quyền xem audit.
- **Dữ liệu đầu vào:** Alert, activity log, thay đổi cấu hình và sự kiện thiết bị.
- **Các bước thao tác:** Mở Audit Logs, lọc theo nhóm sự kiện, tìm kiếm từ khóa, xem thay đổi cấu hình và trạng thái cảnh báo.
- **Kết quả mong đợi:** Giao diện hiển thị dữ liệu từ `activity_logs` và `alerts`; người dùng truy vết được thời điểm và nguồn thay đổi.
- **Thành phần kỹ thuật:** `AuditLogsPage.jsx`, `auditApi.js`, `AuditQueryController.java`, `ActivityLogRepository.java`, `AlertRepository.java`.

Kịch bản 4: Quản lý lịch mode và quyền thành viên

- **Bối cảnh:** Home có nhiều thành viên và cần lịch chuyển mode theo khung giờ.
- **Vai trò người dùng:** Owner hoặc co-owner.
- **Dữ liệu đầu vào:** Lịch mode, vai trò thành viên, quyền kích hoạt profile.
- **Các bước thao tác:** Mở Settings, tạo/sửa mode schedule, cập nhật quyền thành viên, kiểm tra kết quả trên dashboard.
- **Kết quả mong đợi:** Lịch được lưu và scheduler thực thi; quyền thành viên cập nhật trong `home_users`.
- **Thành phần kỹ thuật:** Settings page, `ModeSchedule controller/service`, `HomeUser controller` và `HomeUserManagement service`.

3.9 Đối chiếu hiện thực hóa thiết kế

Các bảng dưới đây đối chiếu các nhóm thiết kế với vị trí hiện thực hóa tương ứng trong dự án. Các đường dẫn backend được rút gọn từ prefix `backend/src/main/java/com/java/` để tránh làm vỡ cột khi trình bày.

Table 4: Đối chiếu hiện thực hóa nhóm cấu hình dự án và triển khai

Nhóm	File/thư mục	Vai trò	Mapping
Docker Compose	<code>docker-compose.yml</code>	Định nghĩa service db, rabbitmq, backend, frontend, port và volume.	Deployment



Nhóm	File/thư mục	Vai trò	Mapping
Backend build	backend/pom.xml	Khai báo Spring Boot, JPA, Security, Flyway, PostgreSQL, RabbitMQ, Mail và test dependency.	Công nghệ
Backend config	backend/src/main/resources/application.properties	Khai báo port, datasource, Flyway, JWT, RabbitMQ, Mail, Telegram và SSE config.	Cấu hình
Backend image	backend/Dockerfile	Build jar bằng Maven và chạy Spring Boot API tại cổng 8080.	Runtime
Frontend build	frontend/package.json	Khai báo React, React Router, Recharts, Axios, Vite và scripts frontend.	Công nghệ
Frontend image	frontend/Dockerfile	Build React/Vite app bằng Node và phục vụ static build.	Runtime
Frontend server	frontend/nginx.conf	Cấu hình Nginx phục vụ frontend trong container.	Deployment

Table 5: Đối chiếu hiện thực hóa nhóm giao diện web

Nhóm	File/thư mục	Vai trò	Mapping
Routing	frontend/src/router/AppRouter.jsx	Định nghĩa route, route guard và luồng điều hướng chính của ứng dụng.	UI layer
Auth state	frontend/src/providers/AuthProvider.jsx	Cung cấp trạng thái xác thực và thông tin người dùng cho frontend.	Provider
Auth storage	frontend/src/utis/authStorage.js	Lưu, đọc và xóa token/session ở phía frontend.	Auth
API client	frontend/src/api/apiClient.js	Gọi backend, gán bearer token, refresh token và chuẩn hóa lỗi API.	Request
Auth API	frontend/src/api/authApi.js	Gửi request đăng nhập, refresh token và lấy thông tin phiên.	Auth
Dashboard API	frontend/src/api/dashboardApi.js	Tải dữ liệu dashboard và phục vụ realtime dashboard.	Dashboard
Config API	frontend/src/api/configApi.js	Gọi backend cho cấu hình tự động hóa và profile active.	Config
Dashboard page	frontend/src/pages/dashboard/DashboardPage.jsx	Màn hình giám sát trạng thái thiết bị và dữ liệu realtime.	Demo
Config page	frontend/src/pages/config/ConfigPage.jsx	Màn hình cấu hình ngưỡng và thiết bị giám sát.	Demo
History page	frontend/src/pages/history/HistoryPage.jsx	Hiển thị lịch sử sensor và biểu đồ dữ liệu.	History
Settings/Audit	frontend/src/pages/settings, frontend/src/pages/audit	Quản lý thành viên, lịch mode và nhật ký hoạt động.	Admin UI
UI components	frontend/src/components	Chứa card, table, modal, form, layout và filter dùng lại.	UI component

Table 6: Đối chiếu hiện thực hóa nhóm backend API

Nhóm	File/thư mục	Vai trò	Mapping
Backend entry	SmartHouseApplication.java	Entry point Spring Boot và bootstrap toàn bộ backend.	Runtime
Auth API	controller/AuthController.java	Nhận request đăng nhập, refresh token và thông tin phiên.	API layer
Dashboard API	controller/DashboardController.java	Cung cấp dashboard summary và stream SSE cho home.	Dashboard
Telemetry API	controller/TelemetryController.java	Nhận telemetry từ gateway hoặc thiết bị.	Telemetry
Control API	controller/ControlController.java	Nhận thao tác điều khiển từ frontend.	Control
Command API	controller/CommandController.java	Cung cấp polling command và ACK cho thiết bị.	Device API
Config API	controller/ConfigController.java	Quản lý cấu hình tự động hóa và profile active.	Config
Alert API	controller/AlertController.java	Truy vấn và cập nhật trạng thái cảnh báo.	Alert
Audit API	controller/AuditQueryController.java	Truy vấn nhật ký hoạt động và bộ lọc audit.	Audit
Device/User/Schedule API	controller/DeviceController.java, HomeUserController.java, ModeScheduleController.java	Quản lý thiết bị, thành viên home và lịch mode.	Management
Security config	config/SecurityConfig.java	Cấu hình JWT filter, CORS, public endpoint và password encoder.	Security
Exception handler	GlobalExceptionHandler.java	Chuẩn hóa response lỗi validation, not found, forbidden và lỗi hệ thống.	Error handling

Table 7: Đối chiếu hiện thực hóa nhóm xử lý nghiệp vụ, sự kiện và lịch

Nhóm	File/thư mục	Vai trò	Mapping
Telemetry service	domain/service/ TelemetryIngestService.java	Điều phối luồng nhận telemetry và kích hoạt xử lý sau lưu.	Service
Telemetry persistence	domain/service/ TelemetryPersistenceService.java	Kiểm tra device/sensor, lưu sensor_data , cập nhật runtime state.	Database
Telemetry automation	domain/service/ TelemetryAutomationService.java	Kích hoạt tự động hóa khi telemetry thuộc cấu hình active.	Automation
Control facade	domain/service/ControlFacadeService.java	Điều phối điều khiển manual và auto.	Facade
Command service	domain/service/ ControlCommandService.java	Tạo, lấy, gửi và xác nhận command cho thiết bị.	Command
Config/Dashboard	domain/service/ConfigService.java, DashboardService.java	Xử lý cấu hình tự động hóa và dữ liệu dashboard.	Use case
Alert/Audit	domain/service/AlertService.java, ActivityLogService.java	Xử lý cảnh báo và nhật ký hoạt động.	Monitoring
Auth strategy	domain/provider/ AuthenticationStrategy.java	Định nghĩa hợp đồng strategy xác thực.	Pattern
Auth providers	domain/provider/ LocalAuthenticationStrategy.java, GoogleAuthenticationStrategy.java	Triển khai xác thực local và Google.	Pattern
Threshold strategy	domain/service/ ThresholdRuleStrategy.java	Định nghĩa hợp đồng kiểm tra ngưỡng sensor.	Pattern
Threshold implementations	TemperatureThresholdStrategy.java, LightThresholdStrategy.java, HumidityThresholdStrategy.java, MotionThresholdStrategy.java	Kiểm tra ngưỡng nhiệt độ, ánh sáng, độ ẩm và chuyển động.	Alert
Event bus	eventing/DomainEventBus.java, SimpleDomainEventBus.java	Publish và dispatch domain event nội bộ.	Observer
Realtime listeners	eventing/ DashboardRealtimeTelemetryListener.java, CommandLongPollListener.java	Phát dashboard realtime và long-poll command.	Realtime
Schedulers	scheduler/OfflineScheduler.java, DeviceScheduleScheduler.java, ManualHoldScheduler.java, ModeAutomationScheduler.java	Job nền cho offline, schedule, manual hold và mode automation.	Scheduler

Table 8: Đối chiếu hiện thực hóa nhóm lưu trữ dữ liệu

Nhóm	File/thư mục	Vai trò	Mapping
Migration chính	backend/src/main/resources/db/ migration/V1__init.sql	Tạo enum, bảng, trigger, constraint, index và seed data.	Schema
Migration bổ sung	V2__add_control_command_source.sql đến V5__audit_dashboard_pagination_indexes.sql	Bổ sung cột source và index phục vụ command, audit, dashboard.	Schema
Device repository	persistence/repo/DeviceRepository.java	Truy vấn thiết bị, trạng thái và phạm vi home.	Repository
Sensor repository	persistence/repo/SensorRepository.java, SensorDataRepository.java	Truy vấn sensor và dữ liệu telemetry.	Repository
Config repository	persistence/repo/ConfigRepository.java	Truy vấn cấu hình và profile active.	Repository
Command repository	persistence/repo/ ControlCommandRepository.java	Truy vấn command pending, sent, acked.	Repository
Audit repository	persistence/repo/ ActivityLogRepository.java, AlertRepository.java	Truy vấn activity log và alert.	Repository
User/Home repository	persistence/repo/UserRepository.java, HomeRepository.java, HomeUserRepository.java	Truy vấn user, home và membership.	Repository
Outbox repository	persistence/repo/ OutboxEventRepository.java	Lưu sự kiện tích hợp cần publish qua RabbitMQ.	Integration
Device entities	persistence/entity/DeviceEntity.java, DeviceCapabilityEntity.java, DeviceRuntimeStateEntity.java	Ảnh xạ bảng thiết bị, capability và trạng thái runtime.	Entity
Sensor entities	persistence/entity/SensorEntity.java, SensorDataEntity.java	Ảnh xạ sensor và dữ liệu telemetry.	Entity
Control entities	persistence/entity/ConfigEntity.java, ControlCommandEntity.java, ScheduleEntity.java	Ảnh xạ cấu hình, command và lịch.	Entity
Identity entities	persistence/entity/UserEntity.java, HomeEntity.java, HomeUserEntity.java	Ảnh xạ người dùng, nhà và thành viên.	Entity
Monitoring entities	persistence/entity/AlertEntity.java, ActivityLogEntity.java, OutboxEventEntity.java	Ảnh xạ cảnh báo, audit và outbox event.	Entity

Table 9: Đối chiếu hiện thực hóa nhóm gateway, thiết bị, mô phỏng và camera

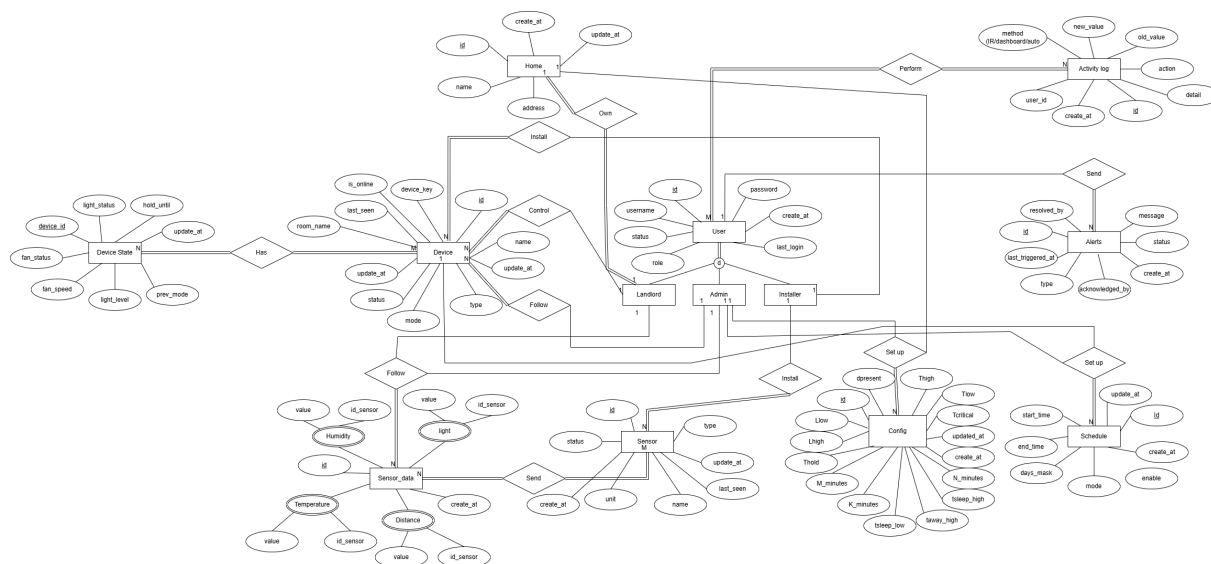
Nhóm	File/thư mục	Vai trò	Mapping
Gateway	gateway.py	Flask gateway chuyển telemetry, command, ACK và camera proxy.	Integration
Simulator	virtual_device_simulator.py	Mô phỏng sensor, command, IR, cửa, quạt và đèn.	Demo
Firmware	aiot_code_v3.py	Mã YoloBit/OhStem đọc sensor, LCD, IR, command và telemetry.	Device
Camera API	camera_interface.py	Flask API dùng OpenCV/YOLO cho nhận diện người/chuyển động.	Camera
YOLO model	yolov8n.pt, yolov8m.pt	Model nhận diện được camera service sử dụng khi chạy.	Asset

Table 10: Đối chiếu hiện thực hóa nhóm tài liệu, kiểm thử và tài nguyên

Nhóm	File/thư mục	Vai trò	Mapping
README	Readme.md	Tài liệu hướng dẫn tổng quan project.	Documentation
API docs	docs/api-design-patterns.md	Ghi chú API và pattern hỗ trợ đối chiếu thiết kế.	Documentation
Schema docs	docs/smart_home_schema_documentation.md	Tài liệu mô tả schema smart home.	Database
Generated diagrams	docs/diagram/report_generated	Chứa source sơ đồ sinh mới cho report.	Diagram
Report images	Images/	Chứa logo, use case, ERD, mapping và ảnh sơ đồ.	Asset
Backend tests	backend/src/test/java	Unit/integration test cho backend.	Test
Gateway tests	scripts/test_gateway_ack_queue.py	Kiểm thử hàng đợi ACK gateway.	Test
Camera tests	scripts/test_camera_check_human_cached.py	Kiểm thử cache API nhận diện người.	Test

4 Database

4.1 ERD



Hình 20: ERD DATABASE

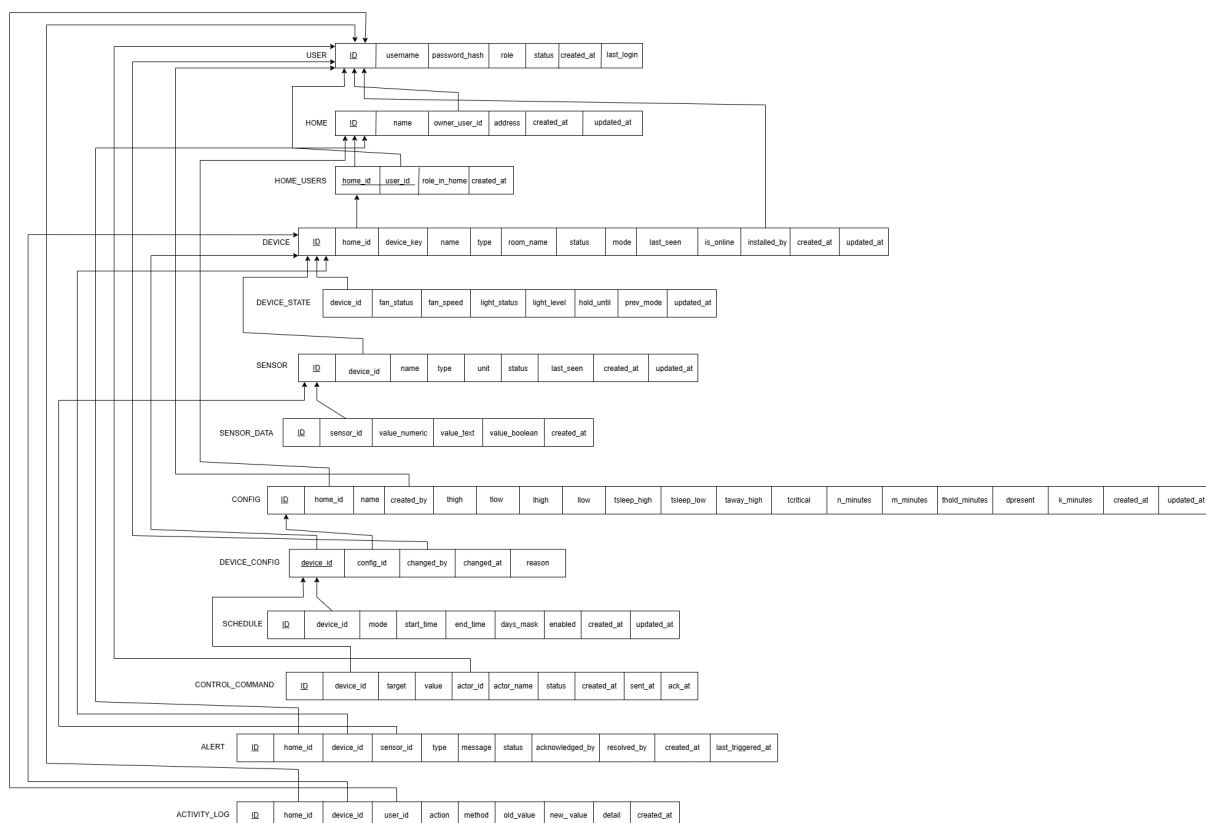
4.2 MAPPING

4.3 Đặc tả chi tiết Cơ sở dữ liệu

Cơ sở dữ liệu của hệ thống Smart House được triển khai trên PostgreSQL và quản lý bằng Flyway migration. Thiết kế dữ liệu xoay quanh năm nhóm chính: định danh và quyền truy cập, nhà và thành viên, thiết bị và capability, telemetry và trạng thái, cấu hình tự động hóa và audit. Điểm quan trọng của mô hình là thiết bị không bị cố định bởi một vài cột trạng thái, mà được mô tả bằng device_capabilities, device_runtime_state, device_state_history và control_commands.

4.3.1 Nhóm thực thể Định danh và Quyền truy cập (Identity & Access)

- **users (Người dùng):**
 - *Khóa chính:* id.
 - *Attributes:* username, password_hash, role, status, must_change_password, invited_at, last_login, created_at, updated_at.
 - *Ý nghĩa:* Lưu tài khoản hệ thống, vai trò toàn cục như ADMIN, CUSTOMER, INSTALLER, SUPER_ADMIN và trạng thái tài khoản.
- **user_auth_providers (Nhà cung cấp xác thực):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* user_id tham chiếu users(id).
 - *Attributes:* provider, provider_user_id, provider_email, linked_at, last_used_at.
 - *Ý nghĩa:* Liên kết tài khoản với phương thức đăng nhập LOCAL hoặc GOOGLE.
- **user_refresh_tokens (Refresh token):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* user_id tham chiếu users(id).
 - *Attributes:* token_hash, issued_at, expires_at, revoked_at, created_by_ip, user_agent.
 - *Ý nghĩa:* Quản lý phiên làm mới đăng nhập và hỗ trợ thu hồi token.



Hình 21: MAPPING DATABASE

4.3.2 Nhóm thực thể Nhà và Thành viên (Home & Membership)

- **homes (Nhà):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* owner_user_id tham chiếu users(id).
 - *Attributes:* name, address, created_at, updated_at.
 - *Ý nghĩa:* Xác định phạm vi logic của một nhà thông minh, làm khóa phân vùng cho thiết bị, cấu hình, cảnh báo và nhật ký.
- **home_users (Thành viên nhà):**
 - *Khóa chính:* home_id, user_id.
 - *Khóa ngoại:* home_id tham chiếu homes(id), user_id tham chiếu users(id).
 - *Attributes:* role_in_home, allow_profile_activation, is_primary, created_at, updated_at.
 - *Ý nghĩa:* Biểu diễn quan hệ nhiều-nhiều giữa người dùng và nhà, đồng thời lưu quyền ở cấp home.

4.3.3 Nhóm thực thể Thiết bị và Năng lực (Device & Capability)

- **devices (Thiết bị):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* home_id tham chiếu homes(id), installed_by tham chiếu users(id).
 - *Attributes:* device_key, name, class, subtype, room_name, status, last_seen, is_online, firmware_version.
 - *Ý nghĩa:* Lưu controller, actuator, sensor node, hub hoặc thiết bị khác trong nhà.
- **device_capabilities (Năng lực thiết bị):**
 - *Khóa chính:* device_id, capability_code.
 - *Khóa ngoại:* device_id tham chiếu devices(id).

- *Attributes*: value_type, is_writable, min_value, max_value, unit, created_at.
- *Ý nghĩa*: Mô tả các khả năng như POWER, SPEED, BRIGHTNESS, MODE, TEMPERATURE, HUMIDITY, MOTION.
- **device_runtime_state** (Trạng thái hiện tại):
 - *Khóa chính*: device_id, capability_code.
 - *Khóa ngoại*: device_id, capability_code tham chiếu device_capabilities.
 - *Attributes*: value_boolean, value_number, value_text, updated_at.
 - *Ý nghĩa*: Lưu giá trị mới nhất của từng capability để dashboard hiển thị nhanh trạng thái hiện tại.
- **device_state_history** (Lịch sử trạng thái):
 - *Khóa chính*: id.
 - *Khóa ngoại*: device_id, capability_code tham chiếu device_capabilities; changed_by tham chiếu users(id).
 - *Attributes*: value_boolean, value_number, value_text, source, source_ref_id, created_at.
 - *Ý nghĩa*: Lưu timeline thay đổi trạng thái phục vụ lịch sử, audit và phân tích nguyên nhân.
- **device_configs** (Cấu hình áp dụng cho thiết bị):
 - *Khóa chính*: device_id.
 - *Khóa ngoại*: device_id tham chiếu devices(id), config_id tham chiếu configs(id), changed_by tham chiếu users(id).
 - *Attributes*: changed_at, reason.
 - *Ý nghĩa*: Ghi nhận cấu hình đang áp dụng cho từng thiết bị, bảo đảm thiết bị và cấu hình thuộc cùng home.

4.3.4 Nhóm thực thể Cảm biến và Dữ liệu (Sensor & Telemetry)

- **sensors** (Cảm biến):
 - *Khóa chính*: id.
 - *Khóa ngoại*: device_id tham chiếu devices(id).
 - *Attributes*: name, sensor_kind, unit, status, last_seen, created_at, updated_at.
 - *Ý nghĩa*: Đại diện cho kênh đo thuộc thiết bị SENSOR_NODE; hệ thống có các loại như TEMPERATURE, HUMIDITY, LIGHT, MOTION, DISTANCE, SMOKE.
- **sensor_data** (Dữ liệu cảm biến):
 - *Khóa chính*: id.
 - *Khóa ngoại*: sensor_id tham chiếu sensors(id).
 - *Attributes*: value_numeric, value_text, value_boolean, created_at.
 - *Ý nghĩa*: Lưu dữ liệu telemetry theo thời gian để phục vụ dashboard, biểu đồ History, cảnh báo và tự động hóa.

4.3.5 Nhóm thực thể Tự động hóa và Điều khiển (Automation & Control)

- **configs** (Cấu hình tự động hóa):
 - *Khóa chính*: id.
 - *Khóa ngoại*: home_id tham chiếu homes(id), created_by tham chiếu users(id); các cột monitoring tham chiếu devices(id).
 - *Attributes*: name, thigh, tlow, lhigh, llow, tsleep_high, tsleep_low, taway_high, tcritical, n_minutes, m_minutes, thold_minutes, dpresent, k_minutes, auto_fan_speed, sleep_fan_speed, away_fan_speed, is_active.
 - *Ý nghĩa*: Định nghĩa ngưỡng nhiệt độ, ánh sáng, thời gian giữ trạng thái, tốc độ quạt và thiết bị giám sát. Ràng buộc unique index bảo đảm mỗi home chỉ có một cấu hình active.
- **schedules** (Lịch thiết bị):

- *Khóa chính:* id.
- *Khóa ngoại:* device_id, capability_code tham chiếu device_capabilities; created_by tham chiếu users(id).
- *Attributes:* value_boolean, value_number, value_text, start_time, end_time, days_mask, enabled, priority.
- *Ý nghĩa:* Thiết lập giá trị cần áp dụng cho capability của thiết bị theo thời gian và ngày trong tuần.
- **control_commands (Lệnh điều khiển):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* device_id tham chiếu devices(id), actor_id tham chiếu users(id).
 - *Attributes:* target, value_boolean, value_number, value_text, actor_name, status, source, created_at, sent_at, ack_at.
 - *Ý nghĩa:* Theo dõi vòng đời lệnh từ PENDING, SENT đến ACKED, FAILED hoặc CANCELLED.

4.3.6 Nhóm thực thể Giám sát và Tích hợp (Monitoring & Integration)

- **alerts (Cảnh báo):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* home_id tham chiếu homes(id), device_id tham chiếu devices(id), sensor_id tham chiếu sensors(id), acknowledged_by và resolved_by tham chiếu users(id).
 - *Attributes:* type, message, status, acknowledged_at, resolved_at, created_at, last_triggered_at.
 - *Ý nghĩa:* Quản lý cảnh báo như CRITICAL_TEMP, DEVICE_OFFLINE, SENSOR_ERROR, HIGH_LIGHT, LOW_LIGHT, MOTION_DETECTED, WRONG_PASSWORD. Vòng đời cảnh báo gồm ACTIVE, ACK, RESOLVED.
- **activity_logs (Nhật ký hoạt động):**
 - *Khóa chính:* id.
 - *Khóa ngoại:* home_id tham chiếu homes(id), device_id tham chiếu devices(id), user_id tham chiếu users(id).
 - *Attributes:* action, method, old_value, new_value, detail, created_at.
 - *Ý nghĩa:* Lưu vết thao tác điều khiển, cấu hình, tự động hóa, cảnh báo và các sự kiện hệ thống để phục vụ audit.
- **outbox_event (Sự kiện tích hợp):**
 - *Khóa chính:* id.
 - *Attributes:* aggregate_type, aggregate_id, event_type, payload, status, retry_count, last_error, created_at, published_at.
 - *Ý nghĩa:* Lưu sự kiện cần phát sang RabbitMQ, hỗ trợ retry và theo dõi trạng thái publish.

4.3.7 Quan hệ dữ liệu chính

- Một users được phép sở hữu nhiều homes và tham gia nhiều home qua home_users.
- Một homes có nhiều devices, configs, alerts và activity_logs.
- Một devices có nhiều device_capabilities, sensors, schedules, control_commands, device_state_history.
- Một capability của thiết bị có tối đa một bản ghi trạng thái hiện tại trong device_runtime_state.
- Một sensors có nhiều sensor_data; dữ liệu này là đầu vào cho dashboard, lịch sử, cảnh báo và tự động hóa.
- Một configs liên kết tới các thiết bị giám sát thông qua monitoring_temperature_device_id, monitoring_humidity_device_id, monitoring_light_sensor_device_id, monitoring_motion_device_id,

monitoring_fan_device_id, monitoring_light_device_id.

- alerts và activity_logs liên kết ngược về home, device, sensor hoặc user để truy vết nguyên nhân.

4.3.8 Các ràng buộc và chỉ mục quan trọng

- device_key là duy nhất, giúp gateway và thiết bị định danh đúng thiết bị.
- Các bảng lưu giá trị như sensor_data, control_commands, schedules, device_runtime_state yêu cầu đúng một kiểu giá trị trong ba cột boolean, number hoặc text.
- Trigger database kiểm tra sensor chỉ gắn với thiết bị SENSOR_NODE, lệnh và runtime state phải khớp capability, alert phải thuộc đúng home của device hoặc sensor.
- Index trên activity_logs, sensor_data, control_commands, alerts hỗ trợ truy vấn dashboard, history, long-poll command và audit.
- Unique index uq_configs_one_active_per_home bảo đảm một home chỉ có một profile active tại một thời điểm.

4.3.9 Cấu hình kết nối, migration và seed data

- Backend kết nối PostgreSQL qua spring.datasource.url, spring.datasource.username, spring.datasource.password trong backend/src/main/resources/application.properties. Các biến môi trường tương ứng là DB_URL, DB_USERNAME, DB_PASSWORD.
- Khi chạy bằng Docker Compose, service db dùng image postgres:18-alpine, expose cổng 5432:5432 và lưu dữ liệu vào volume postgres_data.
- Flyway được bật qua spring.flyway.enabled; thư mục migration là classpath:db/migration.
- V1__init.sql tạo enum, bảng, trigger, index, ràng buộc và dữ liệu mẫu. Các migration V2 đến V5 bổ sung cột source cho lệnh điều khiển và index phục vụ realtime command, activity log, audit dashboard.

4.3.10 Mapping database - entity - repository - service

Table 11: Mapping database, entity, repository và service

Bảng	Entity	Repository	Service liên quan
users	UserEntity.java	UserRepository.java	AuthenticationService, UserProvisioningService, ProfileService
user_auth_providers	UserAuthProviderEntity.java	UserAuthProviderRepository.java	UserAuthProviderLinker, AuthenticationService
user_refresh_tokens	UserRefreshTokenEntity.java	UserRefreshTokenRepository.java	RefreshTokenService, TokenRefreshFacade
homes	HomeEntity.java	HomeRepository.java	HomeAccessService, HomeProfileService
home_users	HomeUserEntity.java	HomeUserRepository.java	HomeUserManagementService, HomeMembershipService
devices	DeviceEntity.java	DeviceRepository.java	DeviceService, DeviceControlService, DashboardService
device_capabilities	DeviceCapabilityEntity.java	DeviceCapabilityRepository.java	DeviceTargetPolicy, DeviceRuntimeStateService
device_runtime_state	DeviceRuntimeStateEntity.java	DeviceRuntimeStateRepository.java	DeviceRuntimeStateService, DashboardRealtimeService
device_state_history	DeviceStateHistoryEntity.java	DeviceStateHistoryRepository.java	DeviceStateHistoryService, TelemetryHistoryService
sensors	SensorEntity.java	SensorRepository.java	TelemetryPersistenceService, SensorSnapshotService
sensor_data	SensorDataEntity.java	SensorDataRepository.java	TelemetryPersistenceService, TelemetryHistoryService
configs	ConfigEntity.java	ConfigRepository.java	ConfigService, TelemetryAutomationService, ModeAutomationService



Bảng	Entity	Repository	Service liên quan
schedules	ScheduleEntity.java	ScheduleRepository.java	ScheduleService, DeviceScheduleScheduler
control_commands	ControlCommandEntity.java	ControlCommandRepository.java	ControlCommandService, DeviceCommandExecutionService
alerts	AlertEntity.java	AlertRepository.java	AlertService, TelemetryAlertService, DeviceOfflineMonitorService
activity_logs	ActivityLogEntity.java	ActivityLogRepository.java	ActivityLogService, AuditDashboardService, AuditFilterService
outbox_event	OutboxEventEntity.java	OutboxEventRepository.java	UserEventOutboxService, OutboxEventPublisher